Log Management System Documentation: Data Structures

Medjitna Belkacem/Lahmar Meriem Imene

May 19, 2025

Contents

1 System Overview

This section provides a high-level overview of the Log Management System, its purpose, and key components.

1.1 Purpose and Scope

This document provides comprehensive documentation for a Log Management System implemented in C. The system is designed to capture, store, manage, and process log entries, offering various functionalities for handling log data efficiently using different data structures and algorithmic techniques. The core purpose of the system is to demonstrate and compare the application of fundamental data structures and algorithms in a practical context.

1.2 Key Components and Data Structures

The system allows users to interact with log data stored in several fundamental data structures:

- **Linked Lists**: Including Singly Linked Lists, Bidirectional Linked Lists, and Circular Linked Lists. These dynamic structures are explored for their flexibility in handling varying numbers of log entries and their performance characteristics for operations like insertion, deletion, and traversal.
- **Queues**: An array-based implementation of a Queue is included, demonstrating First-In, First-Out (FIFO) data management, which could be used for processing logs in the order they are received.
- **Stacks**: An array-based implementation of a Stack is included, demonstrating Last-In, First-Out (LIFO) data management, potentially useful for tasks like undo operations or processing logs in reverse order.
- **Binary Search Trees (BSTs)**: A BST implementation is used to store log entries, ordered by timestamp. This structure facilitates efficient searching, insertion, and deletion operations, particularly useful for quickly retrieving logs within a specific time range.

1.3 Core Functionalities

The system supports a range of operations on these data structures, including:

- Adding new log entries (at the beginning, end, or specific positions).
- Deleting log entries (by ID, position, or timestamp).
- Searching for log entries (by ID or keyword).
- Sorting log entries (by severity or ID).
- Displaying log entries.
- Calculating the number of log entries.
- Converting data between different structures (e.g., Linked List to BST).

1.4 Algorithmic Techniques and Analysis

Additionally, the documentation delves into **Recursion** as an algorithmic technique, showcasing its application in mathematical calculations (factorial, Fibonacci) and linked list manipulation (reversal, finding maximum ID). A significant part of this documentation is dedicated to the **Algorithmic Complexity Analysis** of the implemented operations across all data structures. This analysis, using Big-O notation, evaluates the time and space efficiency of each function, providing insights into how the performance scales with the size of the log data. Understanding these complexities is vital for choosing the most appropriate data structure and algorithm for specific log management tasks and for predicting system performance under different loads.

1.5 Other Essential Aspects

The document also covers essential aspects like **Memory Management**, ensuring efficient allocation and deallocation of memory to prevent leaks; **Error Handling**, outlining mechanisms to deal with potential issues like memory allocation failures or invalid user input; and the **Testing Methodology** used to ensure the system's reliability and correctness.

In summary, this Log Management System serves as a practical exploration of fundamental data structures and algorithms, providing a foundation for understanding their implementation, application, and performance characteristics in the context of managing log data.

Singly Linked Lists

2 Introduction to Linked Lists

Linked lists are fundamental linear data structures where elements are not stored at contiguous memory locations. Instead, each element is a separate object, called a node, that contains a data part and a pointer to the next node in the sequence. This structure allows for efficient insertion and deletion of elements compared to arrays, where these operations can be costly as they may require shifting elements.

Singly linked lists are the simplest type of linked list, where each node contains a data part and a pointer to the next node.

In the context of a log management system, singly linked lists are a suitable choice for storing log entries due to their dynamic size and efficient insertion/deletion at the ends.

3 Log Entry Structure (Singly Linked List)

The basic building block for storing log information in a singly linked list is the `LogEntry` structure.

```
struct LogEntry {
      int id;                              // Auto-incremented identifier
      char message[100];                 // Log content
      struct LogEntry *next;           // Pointer to next entry
      char severity[10];                 // INFO/WARNING/ERROR
      char timestamp[50];                // Formatted timestamp
};
typedef struct LogEntry LogEntry;
```

Structure Breakdown:

‘id‘: An integer to uniquely identify each log entry. ‘message‘: A character array to store the log message content. A fixed size of 100 is used. ‘next‘: A pointer of type ‘struct LogEntry *‘ that points to the subsequent log entry in the list. ‘severity‘: A character array to store the severity level of the log (e.g., ”INFO”, ”WARNING”, ”ERROR”). A fixed size of 10 is used. ‘timestamp‘: A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used.

The ‘typedef‘ statement creates an alias ‘LogEntry‘ for ‘struct LogEntry‘.

4 Singly Linked List Implementation Details

4.1 Creating a New Log Entry Node

The `CreateLogEntry` function allocates memory for a new `LogEntry` node and initializes its fields.

```
#include <stdlib.h> // For malloc and free
#include <stdio.h>   // For printf, fgets, scanf
#include <string.h> // For strncpy, strlen, strcmp, strstr
#include <stdbool.h> // For bool type
#include <time.h>     // For time functions (assuming getCurrentTimeString uses these)

// Assume ID is a global variable defined elsewhere, e.g., int ID = 0;
extern int ID;
// Assume getCurrentTimeString() is defined elsewhere and returns a dynamically allocated string.
extern char* getCurrentTimeString();
// Assume isValidDateTime is defined elsewhere
extern bool isValidDateTime(int year, int month, int day, int hour, int minutes, int seconds);


LogEntry* CreateLogEntry(){
      LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
      if (temp == NULL) {
            printf("\nMemory Allocation Failed\n");
            return NULL;
      }

      temp->id = ID++; // Assuming ID is a global counter
      temp->next = NULL;
      char msg[100];
      char msg2[100]; // Used for message input
      char* temp3 = getCurrentTimeString(); // Function to get timestamp

      if (temp3) {
            strncpy(temp->timestamp, temp3, sizeof(temp->timestamp) - 1);
            temp->timestamp[sizeof(temp->timestamp) - 1] = '\0'; // Ensure null-termination
            free(temp3); // Free memory allocated by getCurrentTimeString
      } else {
            strncpy(temp->timestamp, "ERROR", sizeof(temp->timestamp) - 1);
            temp->timestamp[sizeof(temp->timestamp) - 1] = '\0';
      }

      printf("\nWhat is the message?\n");
      fgets(msg2, sizeof(msg2), stdin);
      size_t len = strlen(msg2);
      if (len > 0 && msg2[len - 1] == '\n') {
            msg2[len - 1] = '\0'; // Remove trailing newline
      }
      strncpy(temp->message, msg2, sizeof(temp->message) - 1);
      temp->message[sizeof(temp->message) - 1] = '\0';

      do {
            printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
            if (fgets(msg, sizeof(msg), stdin) != NULL) {
                  size_t len = strlen(msg);
                  if (len > 0 && msg[len - 1] == '\n') {
                        msg[len - 1] = '\0';
```

```
                }
                strncpy(temp->severity, msg, sizeof(temp->severity) - 1);
                temp->severity[sizeof(temp->severity) - 1] = '\0';
            } else {
                printf("Error reading severity level.\n");
                free(temp); // Free allocated memory on input error
                return NULL;
            }
        } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING")
    != 0 && strcmp(temp->severity, "ERROR") != 0);

        return temp;
}
```

Functionality:

Allocates memory for a new `LogEntry` using `malloc`. Increments a global `ID` counter and assigns it to the new entry's `id`. Sets the `next` pointer to `NULL`. Calls `getCurrentTimeString` to get the current timestamp string and copies it to the `timestamp` field using `strncpy` for safety. Prompts the user for the log message and severity level, reading input using `fgets` and handling potential newline characters. Validates the entered severity level to be one of "INFO", "WARNING", or "ERROR". Returns a pointer to the newly created `LogEntry` node, or `NULL` if memory allocation or input failed.

4.2 Checking if the List is Empty

The `IsLogsEmpty` function provides a simple check to see if the singly linked list is currently empty.

```
bool IsLogsEmpty(LogEntry*L){
      return(L==NULL);
}
```

Functionality:

Takes the head pointer of the list (`L`) as input. Returns `true` if the head pointer is `NULL` (indicating an empty list), and `false` otherwise.

4.3 Adding a Log Entry at the Beginning

The `AddLogEntryAtTheBeginning` function inserts a new log entry at the front of the singly linked list.

```
void AddLogEntryAtTheBeginning(LogEntry**L){
      bool v=IsLogsEmpty(*L);
      LogEntry*temp=CreateLogEntry();

      if (temp==NULL)
      {
            printf("Allocation of Mem failed\n");
            return;
      }

      if (v)
      {
            (*L)=temp; // New node becomes the head
      }
      else
      {
            temp->next=(*L); // New node's next points to the current head
            (*L)=temp; // New node becomes the new head
      }
}
```

Functionality:

Calls `CreateLogEntry` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, the new node's `next` pointer is set to the current head, and the list's head pointer is updated to point to the new node.

4.4 Adding a Log Entry at the End

The `AddLogEntryAtTheEnd` function appends a new log entry to the end of the singly linked list.

```
void AddLogEntryAtTheEnd(LogEntry**L){
      bool v=IsLogsEmpty(*L);
      LogEntry*temp=CreateLogEntry();

      if (temp==NULL)
      {
            printf("Addition Failed\n"); // More specific error message
            return;
      }

      if (v)
      {
            (*L)=temp; // New node becomes the head if list is empty
      }
      else
      {
            LogEntry*Tem=(*L);
            while (Tem->next!=NULL)
            {
                  Tem=Tem->next; // Traverse to the last node
            }
            Tem->next=temp; // Last node's next points to the new node
      }
}
```

Functionality:

Calls `CreateLogEntry` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, it traverses the list to find the last node (the one whose `next` pointer is `NULL`), and then sets the last node's `next` pointer to the new node.

4.5 Inserting a Log Entry at a Specific Position

The `InsertLogEntry` function inserts a new log entry at a specified position within the singly linked list. `InsertLogEntryByPos` is a helper to get the position from the user.

```
void InsertLogEntry(LogEntry** L, int pos) {
      int counter = 1;
      // Create the new log entry using the dedicated function
      LogEntry* temp = CreateLogEntry();
      if (temp == NULL) {
            printf("\nFailed to create new log entry.\n");
            return;
      }

      if (pos == 1 || *L == NULL) {
            temp->next = *L;
            *L = temp; // Insert at the beginning
            return;
      }

      LogEntry* current = *L;
      LogEntry* previous = NULL;

      // Traverse to the node before the desired position
      while (current != NULL && counter < pos) {
            previous = current;
            current = current->next;
            counter++;
      }

      if (counter == pos) {
            // Insert at the specified position
            temp->next = current;
            if (previous != NULL) {
                  previous->next = temp;
            } else {
                  // This case should ideally be covered by pos == 1, but as a safeguard
    :
```

```
                    *L = temp;
            }
      } else {
            // Position is out of range (beyond the end of the list)
            printf("The position is out of range.\n");
            free(temp); // Free allocated memory if position is invalid
      }
}

void InsertLogEntryByPos(LogEntry** L) {
      int pos;
      printf("Enter the position: ");
      scanf("%d", &pos);
      getchar();   // Consume leftover '\n'
      InsertLogEntry(L, pos);
}
```

Functionality:

Takes the head pointer of the list and the desired insertion position as input. Handles insertion at the beginning as a special case. For other positions, it traverses the list to find the node *before* the insertion point. Updates the `next` pointers of the previous node and the new node to insert the new node into the list. If the specified position is beyond the end of the list, it prints an error message and frees the allocated memory for the new node.

4.6 Displaying All Log Entries

The `DisplayLogs` function iterates through the singly linked list and prints the details of each log entry.

```
void displayLogHeader() {
      printf("%-5s | %-32s                                | %-10s | %-100s\n",
                  "ID", "Timestamp", "Severity", "Message");
      printf("
    ------|--------------------------------------------------|------------|-------------------------
    n");
}

void displayLogEntryColumn(LogEntry *entry) {
      printf("%-5d | %-32s | %-10s | %-100s\n",
                  entry ->id, entry->timestamp, entry->severity, entry->message);
}

void DisplayLogs(LogEntry* L) {
      if (L == NULL) {
            printf("Logs are empty\n");
            return;
      }
      displayLogHeader(); // Print header before displaying logs
      while (L != NULL) {
            displayLogEntryColumn(L); // Display details of current log entry
            L = L->next; // Move to the next log entry
      }
}
```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty and prints a message if it is. If the list is not empty, it prints a header for the log entries. It then traverses the list from the head, printing the formatted details of each `LogEntry` using `displayLogEntryColumn` until the end of the list is reached (`NULL`).

4.7 Getting the Number of Log Entries

The `LogSize` function calculates and returns the total number of log entries in the singly linked list.

```
int LogSize(LogEntry*L){
      bool v=IsLogsEmpty(L);

      if (v)
      {
            return 0; // Return 0 if the list is empty
```

```
        }
        else
        {
                int counter=0;
                while (L!=NULL)
                {
                        counter++; // Increment counter for each node
                        L=L->next; // Move to the next node
                }
                return counter; // Return the total count
        }
}
```

Functionality:

Takes the head pointer of the list as input. Returns 0 if the list is empty. Otherwise, it traverses the list from the head, incrementing a counter for each node until the end of the list is reached. Returns the final count.

4.8 Deleting a Log Entry by ID

The `DelLogEntryByID` function searches for a log entry with a specific ID and removes it from the singly linked list.

```
void DelLogEntryByID(LogEntry**L,int ID){
        LogEntry *p; // Pointer to traverse the list
        LogEntry *q; // Pointer to the node to be deleted
        p=*L;

        // Case 1: Deleting the head node
        if(p != NULL && p->id == ID){
                *L = p->next; // Head moves to the next node
                free(p); // Free the old head
                printf("Log with ID %d deleted.\n", ID);
                return;
        }

        // Case 2: Deleting a node other than the head
        while(p != NULL && p->next != NULL){
                if(p->next->id == ID){
                q = p->next; // Node to be deleted
                p->next = p->next->next; // Skip over the node to delete
                free(q); // Free the node
                printf("Log with ID %d deleted.\n", ID);
                return ; // Exit after deletion
                }
                p=p->next; // Move to the next node
        }

        // Case 3: ID not found
        printf("\nID %d NOT found\n", ID);
}
```

Functionality:

Takes the head pointer of the list and the target ID as input. Checks if the head node has the target ID and handles deletion if it does. Otherwise, it traverses the list, keeping track of the current node (`p`). If the `next` node of the current node has the target ID, it updates the `next` pointer of the current node to bypass the node to be deleted and then frees the memory of the deleted node. If the end of the list is reached without finding the ID, it prints a "Not found" message.

4.9 Deleting the First Log Entry

The `DelLogAtTheBeginning` function removes the first log entry from the singly linked list.

```
void DelLogAtTheBeginning(LogEntry**L){
        bool v=IsLogsEmpty(*L);

        if (v)
```

```
5       {
6               printf("They Are empty\n"); // Cannot delete from an empty list
7               return;
8       }
9       else
10      {
11              LogEntry*temp=(*L); // Store the current head
12              (*L)=(*L)->next; // Head moves to the next node
13              free(temp); // Free the memory of the old head
14              printf("First log deleted.\n");
15              return;
16      }
17  }
```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty. If the list is not empty, it updates the head pointer to the second node and frees the memory of the original head node.

4.10 Deleting the Last Log Entry

The `DelLogAtTheEnd` function removes the last log entry from the singly linked list.

```
1   void DelLogAtTheEnd(LogEntry**L){
2         bool v=IsLogsEmpty(*L);
3
4         if (v)
5         {
6               printf("They Are empty\n"); // Cannot delete from an empty list
7               return;
8         }
9         // Special case: only one node in the list
10        else if ((*L)->next==NULL)
11        {
12              free((*L)); // Free the single node
13              (*L)=NULL; // List becomes empty
14              printf("Last log deleted.\n");
15              return;
16        }
17        else
18        {
19              LogEntry*Tem=(*L);
20              // Traverse to the second to last node
21              while (Tem->next->next!=NULL)
22              {
23                    Tem=Tem->next;
24              }
25              LogEntry*fre=Tem->next; // The last node to be freed
26              Tem->next=NULL; // The second to last node becomes the new last node
27              free(fre); // Free the original last node
28              printf("Last log deleted.\n");
29              return;
30        }
31  }
```

Functionality:

Takes the head pointer of the list as input. Checks if the list is empty. Handles the special case where there is only one node in the list. Otherwise, it traverses the list to find the second-to-last node. It then sets the `next` pointer of the second-to-last node to `NULL` and frees the memory of the original last node.

4.11 Deleting a Log Entry by Timestamp

The `DelLogByTimeStamp` function searches for a log entry with a timestamp matching user input and removes it.

```
1   bool isValidDateTime(int year, int month, int day,int hour, int minutes, int seconds) {
2         // Validate year (assuming reasonable range)
3         if (year < 1900 || year > 2100) return false;
```

```
4
5       // Validate month
6       if (month < 1 || month > 12) return false;
7
8       // Validate day
9       if (day < 1 || day > 31) return false;
10
11      // Check months with 30 days
12      if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
13            return false;
14      if (hour>=24||hour<0) // Corrected hour validation
15      {
16            return false;
17      }
18
19      // Check February (ignoring leap years for simplicity)
20      // A more robust check would account for leap years.
21      if (month == 2) {
22            // Basic check, doesn't handle leap years
23            if (day > 28) return false;
24      }
25
26      // Validate minutes and seconds
27      if (minutes < 0 || minutes > 59) return false;
28      if (seconds < 0 || seconds > 59) return false;
29
30      return true;
31  }
32
33  void DelLogByTimeStamp(LogEntry** L) {
34        if (*L == NULL) {
35              printf("Log list is empty!\n");
36              return;
37        }
38        int year,month,day,hour,minutes,seconds;
39        char tryAgain;
40        char temp [50];
41        printf("Enter the TimeStamp in this format:YYYY MM DD HH MM SS\n"); // Adjusted
      prompt
42        do {
43              printf("Enter date and time (YYYY MM DD HH MM SS): ");
44              if (scanf("%d %d %d %d %d %d", &year, &month, &day,&hour, &minutes, &seconds
      ) != 6) {
45                    printf("Invalid input format. Please enter numbers.\n");
46                    while (getchar() != '\n'); // Clear input buffer
47                    continue;
48              }
49
50              while (getchar() != '\n'); // Clear input buffer
51
52              if (!isValidDateTime(year, month, day,hour, minutes, seconds)) {
53                    printf("Invalid date or time values. Please try again.\n");
54                    continue;
55              }
56
57              // Format the entered components into a string.
58              // IMPORTANT: This format needs to MATCH the format used by
      getCurrentTimeString (%Y-%m-%d %H:%M:%S).
59              sprintf(temp,"%04d-%02d-%02d %02d:%02d:%02d", year, month, day, hour,
      minutes, seconds);
60
61              printf("Searching for logs with timestamp containing: %s\n", temp);
62
63              printf("Do you want to enter another date? (y/n): ");
64              scanf(" %c", &tryAgain);
65
66              while (getchar() != '\n');
67
68        } while (tryAgain == 'y' || tryAgain == 'Y');
69
70        LogEntry *current = *L;
71        LogEntry *previous = NULL;
72
```

```
73        while (current != NULL) {
74              if (strstr(current->timestamp, temp) != NULL) { // Using strstr for
    substring match
75                    if (previous != NULL) {
76                          previous->next = current->next;
77                    } else {
78                          *L = current->next; // Deleting the head
79                    }
80
81                    free(current);
82                    printf("Log with timestamp containing '%s' deleted.\n", temp);
83                    return; // Assuming only the first match is deleted
84              }
85              previous = current;
86              current = current->next;
87        }
88
89        printf("No log found with timestamp containing '%s'\n", temp);
90  }
```

Functionality:

Prompts the user for date and time components and validates the input. Formats the input into a string for comparison. Traverses the list and uses ‘strstr‘ to find a log entry whose timestamp contains the formatted input string. If a match is found, it updates the ‘next‘ pointer of the previous node to remove the current node. Handles the case of deleting the head node. Frees the memory of the deleted node. Prints a ”Not found” message if no match is found.

4.12 Sorting Logs by Severity (Distribution Sort)

The `SortBySeverity` function sorts the singly linked list based on the severity level of the log entries (”INFO”, ”WARNING”, ”ERROR”). It uses a counting or distribution sort approach.

```
1   void SortBySeverity(LogEntry** head) {
2         if (*head == NULL) return;
3
4         // Pointers for the heads and tails of the three severity lists
5         LogEntry *info_head = NULL, *info_tail = NULL;
6         LogEntry *warn_head = NULL, *warn_tail = NULL;
7         LogEntry *error_head = NULL, *error_tail = NULL;
8
9         LogEntry* current = *head;
10        while (current != NULL) {
11              LogEntry* next = current->next; // Store the next node before modifying
    current->next
12              current->next = NULL; // Detach the current node from the original list
13
14              if (strcmp(current->severity, "INFO") == 0) {
15                    if (info_head == NULL) {
16                          info_head = current;
17                          info_tail = current;
18                    } else {
19                          info_tail->next = current;
20                          info_tail = current;
21                    }
22              }
23              else if (strcmp(current->severity, "WARNING") == 0) {
24                    if (warn_head == NULL) {
25                          warn_head = current;
26                          warn_tail = current;
27                    } else {
28                          warn_tail->next = current;
29                          warn_tail = current;
30                    }
31              }
32              else { // Assuming anything else is ERROR
33                    if (error_head == NULL) {
34                          error_head = current;
35                          error_tail = current;
36                    } else {
```

```
                        error_tail->next = current;
                        error_tail = current;
                  }
            }
            current = next; // Move to the next node in the original list
      }

      // Reconnect the three lists in the desired order: INFO -> WARNING -> ERROR
      *head = NULL; // Start with an empty list
      LogEntry** tail = head;

      if (info_head) {
            *tail = info_head;
            tail = &info_tail->next;
      }

      if (warn_head) {
            *tail = warn_head;
            tail = &warn_tail->next;
      }

      if (error_head) {
            *tail = error_head;
      }
}
```

Functionality:

Traverses the original linked list. For each node, it checks its severity level ("INFO", "WARNING", or "ERROR"). It moves each node to one of three separate lists: an INFO list, a WARNING list, and an ERROR list. After processing all nodes, it concatenates the three lists in the order INFO, then WARNING, then ERROR, by updating the `next` pointers of the tails of the preceding lists.

This is a stable sort, meaning the relative order of logs with the same severity is preserved.

4.13 Reversing the Linked List

The `ReverseLogs` function reverses the order of the nodes in the singly linked list.

```
void ReverseLogs(LogEntry**H){
      // If the list is empty or has only one node, no reversal is needed
      if (IsLogsEmpty(*H) || (*H)->next == NULL)
      {
            return;
      }

      LogEntry* L1 = *H; // Pointer to the current node (initially head)
      LogEntry* L2 = L1->next; // Pointer to the next node
      L1->next = NULL; // The original head will become the new tail, so its next is
    NULL
      LogEntry* L3 = NULL; // Pointer to the node after L2

      while (L2 != NULL)
      {
            L3 = L2->next; // Store the next node before changing L2->next
            L2->next = L1; // Reverse the pointer: L2's next points to L1
            L1 = L2; // Move L1 forward to the current node (which was L2)
            L2 = L3; // Move L2 forward to the next node (which was stored in L3)
      }
      *H = L1; // L1 is now pointing to the new head (the original last node)
}
```

Functionality:

Uses three pointers (`L1`, `L2`, `L3`) to keep track of the current, next, and next-next nodes during traversal. It iteratively changes the `next` pointer of each node to point to its *previous* node. The original head's `next` pointer is set to `NULL`. The head pointer of the list is updated to point to the original last node, which becomes the new head.

4.14 Searching for a Log Entry by ID

The `serchLogById` function searches for a log entry with a specific ID and prints its details if found.

```
LogEntry* serchLogById(LogEntry*L,int ID){
      if (L == NULL)
      {
            printf("it is empty\n"); // List is empty, ID cannot be found
            return NULL;
      }
      while (L != NULL)
      {
            if (L->id == ID)
            {
                  printf("it was found\n");
                  displayLogEntryColumn(L); // Display the found log entry
                  return L; // Return pointer to the found node
            }
            L = L->next; // Move to the next node
      }
      printf("Not found \n"); // ID not found after traversing the list
      getchar(); // Consume leftover newline (might not be needed depending on prior
    input)
      return NULL; // Return NULL if not found
}
```

Functionality:

Takes the head pointer of the list and the target ID as input. Traverses the list from the head. Compares the `id` of each node with the target ID. If a match is found, it prints a "found" message, displays the log entry details, and returns a pointer to the found node. If the end of the list is reached without finding the ID, it prints a "Not found" message and returns `NULL`.

4.15 Generating Multiple Log Entries

The `GenLog` function allows the user to generate a specified number of log entries and add them to the singly linked list. It uses `InsertLogEntryByPos` to add logs, which in turn calls `InsertLogEntry`.

```
void GenLog(LogEntry** L, int size) {
    int i;
      for (i = 0; i < size; i++) {
            // Insert each new log. The position input will be handled inside
    InsertLogEntryByPos.
            printf("\nEntering details for log %d:\n", i + 1);
            InsertLogEntryByPos(L);

      }
      // Display all logs after generation is complete
      printf("\n--- All Generated Logs ---\n");
      DisplayLogs(*L);
}
```

Functionality:

Takes the head pointer of the list and the number of logs to generate as input. Uses a loop to repeatedly call `InsertLogEntryByPos` to create and insert the specified number of log entries. After generating all logs, it calls `DisplayLogs` to show the contents of the list.

Bidirectional Linked Lists

5 Introduction

Bidirectional linked lists, also known as doubly linked lists, are a more complex form of linked list compared to singly linked lists. Each node in a bidirectional linked list contains three parts:

The data of the node (the log entry details). A pointer (‘next‘) that points to the next node in the sequence. A pointer (‘prev‘) that points to the previous node in the sequence.

This additional ‘prev‘ pointer allows for traversal in both forward and backward directions, which can simplify the implementation of certain operations, such as deletion and reversal, compared to singly linked lists.

6 Log Entry Structure (Bidirectional Linked List)

The structure used for log entries in the bidirectional linked list is `BLogEntry`.

```
struct BLogEntry {
      int id;                              // Auto-incremented identifier
      char message[100];                  // Log content
      struct BLogEntry *next;            // Pointer to next entry
      char severity[10];                  // INFO/WARNING/ERROR
      char timestamp[50];                 // Formatted timestamp
      struct BLogEntry *prev;            // Pointer to previous entry
};
typedef struct BLogEntry BLogEntry;
```

Structure Breakdown:

‘id‘: An integer to uniquely identify each log entry. ‘message‘: A character array to store the log message content. A fixed size of 100 is used. ‘next‘: A pointer that points to the subsequent log entry in the list. ‘severity‘: A character array to store the severity level of the log (e.g., ”INFO”, ”WARNING”, ”ERROR”). A fixed size of 10 is used. ‘timestamp‘: A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used. ‘prev‘: A pointer that points to the previous log entry in the list.

The ‘typedef‘ statement creates an alias ‘BLogEntry‘ for ‘struct BLogEntry‘.

7 Bidirectional Linked List Implementation Details

7.1 Creating a New Bidirectional Log Entry Node

The `CreateBLogEntryBI` function allocates memory for a new `BLogEntry` node and initializes its fields, including the ‘prev‘ pointer.

```
BLogEntry*CreateBLogEntryBI(){
    BLogEntry* temp = (BLogEntry*)malloc(sizeof(BLogEntry));
    if (temp == NULL) {
        printf("\nMemory Allocation Failed\n");
        return NULL;
    }

    temp->id = ID++;
    temp->next = NULL;
    temp->prev = NULL; // Initialize prev pointer
    char msg[100];
    char msg2[100];
    char* temp3 = getCurrentTimeString();

    if (temp3) {
        strcpy(temp->timestamp, temp3);
        free(temp3);
    } else {
        strcpy(temp->timestamp, "ERROR");
    }

    printf("\nWhat is the message?\n");
    fgets(msg2, sizeof(msg2), stdin);
    size_t len = strlen(msg2);
    if (len > 0 && msg2[len - 1] == '\n') {
        msg2[len - 1] = '\0';
    }
    strcpy(temp->message, msg2);

    do {
        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
        if (fgets(msg, sizeof(msg), stdin) != NULL) {
```

```
            size_t len = strlen(msg);
            if (len > 0 && msg[len - 1] == '\n') {
                msg[len - 1] = '\0';
            }
            strcpy(temp->severity, msg);
        } else {
            printf("Error reading severity level.\n");
            free(temp);
            return NULL;
        }
    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
     0 && strcmp(temp->severity, "ERROR") != 0);
    return temp;
}
```

Functionality:

Allocates memory for a new `BLogEntry` using `malloc`. Increments a global `ID` counter and assigns it to the new entry's `id`. Sets both the `next` and `prev` pointers to `NULL`. Calls `getCurrentTimeString` to get the current timestamp string and copies it to the `timestamp` field. Prompts the user for the log message and severity level, reading input using `fgets` and handling potential newline characters. Validates the entered severity level. Returns a pointer to the newly created `BLogEntry` node, or `NULL` if memory allocation or input failed.

7.2 Checking if the Bidirectional List is Empty

The `IsLogsEmpty` function (reused for both list types) checks if the bidirectional linked list is empty.

```
bool IsLogsEmpty(BLogEntry*L){
    return(L==NULL);
}
```

Functionality:

Takes the head pointer of the list (`L`) as input. Returns `true` if the head pointer is `NULL`, and `false` otherwise.

7.3 Adding a Log Entry at the Beginning (Bidirectional)

The `AddBLogEntryAtTheBeginning` function inserts a new log entry at the front of the bidirectional linked list.

```
void AddBLogEntryAtTheBeginning(BLogEntry**L){

    bool v=IsLogsEmpty(*L);

    BLogEntry*temp=CreateBLogEntryBI();

    if (temp==NULL)
    {
        printf("Allocation of Mem failed\n");
        return;
    }

    if (v)
    {
        (*L)=temp;
        return;
    }
    else
    {
        temp->next=(*L);
        (*L)->prev = temp; // Update prev pointer of the original head
        (*L)=temp;
        return;
    }
}
```

Functionality:

Calls `CreateBLogEntryBI` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, the new node's `next` pointer is set to the current head, the original head's `prev` pointer is set to the new node, and the list's head pointer is updated to the new node.

7.4 Adding a Log Entry at the End (Bidirectional)

The `AddBLogEntryAtTheEndBI` function appends a new log entry to the end of the bidirectional linked list.

```
void AddBLogEntryAtTheEndBI(BLogEntry**L){
    bool v=IsLogsEmpty(*L);

    BLogEntry*temp=CreateBLogEntryBI();

    if (temp==NULL)
    {
        printf("Addition Failed\n");
        return;
    }

    if (v)
    {
        (*L)=temp;
    }

    else
    {
        BLogEntry*Tem=(*L);
        while (Tem->next!=NULL)
        {
            Tem=Tem->next;
        }
        Tem->next=temp;
        temp->prev=Tem; // Update prev pointer of the new node
        return;
    }
}
```

Functionality:

Calls `CreateBLogEntryBI` to get a new log entry node. If the list is empty, the new node becomes the head. If the list is not empty, it traverses the list to find the last node, sets the last node's `next` pointer to the new node, and sets the new node's `prev` pointer to the original last node.

7.5 Inserting a Log Entry at a Specific Position (Bidirectional)

The `InsertBLogEntryBI` function inserts a new log entry at a specified position within the bidirectional linked list. `InsertBLogEntryByPosBI` is a helper to get the position from the user.

```
void InsertBLogEntryBI(BLogEntry** L, int pos) {
    int counter = 1;
    BLogEntry* temp = (BLogEntry*)malloc(sizeof(BLogEntry));
    if (temp == NULL) {
        printf("\nMemory Allocation Failed\n");
        return;
    }

    temp->id = ID++;
    temp->next = NULL;
    temp->prev = NULL; // Initialize prev pointer
    char msg[100];
    char msg2[100];
    char* temp3 = getCurrentTimeString();

    if (temp3) {
        strcpy(temp->timestamp, temp3);
        free(temp3);
    } else {
```

```
        strcpy(temp->timestamp, "ERROR");
    }

    printf("\nWhat is the message?\n");
    fgets(msg2, sizeof(msg2), stdin);
    size_t len = strlen(msg2);
    if (len > 0 && msg2[len - 1] == '\n') {
        msg2[len - 1] = '\0';
    }
    strcpy(temp->message, msg2);

    do {
        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
        if (fgets(msg, sizeof(msg), stdin) != NULL) {
            size_t len = strlen(msg);
            if (len > 0 && msg[len - 1] == '\n') {
                msg[len - 1] = '\0';
            }
            strcpy(temp->severity, msg);
        } else {
            printf("Error reading severity level.\n");
            free(temp);
            return;
        }
    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
     0 && strcmp(temp->severity, "ERROR") != 0);

    if (pos == 1 || *L == NULL) {
        temp->next = *L;
        if (*L != NULL) {
            (*L)->prev = temp; // Update prev of original head
        }
        temp->prev=NULL; // New head has no previous
        *L = temp;
        return;
    }

    BLogEntry* current = *L;
    BLogEntry* previous = NULL;

    while (current != NULL && counter < pos) {
        previous = current;
        current = current->next;
        counter++;
    }

    if (counter == pos) {
        temp->next = current;
        temp->prev = previous;
        if (previous != NULL) {
            previous->next = temp;
        }
        if (current != NULL) {
            current->prev = temp; // Update prev of the node after insertion point
        }
    } else {
        printf("The position is out of range.\n");
        free(temp);
    }
}

void InsertBLogEntryByPosBI(BLogEntry** L) {
    int pos;
    printf("Enter the position: ");
    scanf("%d", &pos);
    getchar();  // Consume leftover '\n'
    InsertBLogEntryBI(L, pos);
}
```

Functionality:

Calls `CreateBLogEntryBI` to get a new log entry node. Handles insertion at the beginning as a special case, updating the `prev` pointer of the original head. For other positions, it traverses

the list to find the node *before* the insertion point. Updates the `next` pointer of the previous node and the `prev` pointer of the current node (the one after the insertion point) to link the new node into the list. If the specified position is out of range, it prints an error and frees the allocated memory.

7.6 Displaying All Log Entries (Bidirectional)

The `DisplayLogs` function (overloaded or a separate function in practice) iterates through the bidirectional linked list and prints the details of each log entry. The provided code includes a version that takes a 'pos' argument, likely for highlighting a specific entry during navigation.

```
void displayBLogEntryColumn(BLogEntry *entry) {
    if (entry==NULL)
    {
        printf("it is Empty \n");
        return;
    }
    printf("%-5d | %-32s | %-10s | %-100s\n",
           entry->id, entry->timestamp, entry->severity, entry->message);
}
void displaySelBLogEntryColumn(BLogEntry *entry) {
    printf("%-5d | %-32s | %-10s | %-100s\n",
           entry->id, entry->timestamp, entry->severity, entry->message);
}

void DisplayLogs(BLogEntry* L,int pos) { % Note: This function name conflicts with the
    singly list version
    if (L == NULL) {
        printf("Logs are empty\n");
        return;
    }
    displayLogHeader();
    int counter=1;
    while (L != NULL) {
        if (pos==counter)
        {
            displaySelBLogEntryColumn(L); % Highlight selected entry
        }
        else
        {
            displayBLogEntryColumn(L); % Display normally
        }
        L = L->next;
        counter++;
    }
}
```

Functionality:

Takes the head pointer of the list and an optional position to highlight as input. Checks if the list is empty. Prints a header. Traverses the list, printing the details of each entry. If the current entry's position matches the 'pos' argument, it uses 'displaySelBLogEntryColumn' for highlighting.

7.7 Getting the Number of Log Entries (Bidirectional)

The `LogSize` function (reused) calculates the size of the bidirectional linked list.

```
int LogSize(BLogEntry*L){

    bool v=IsLogsEmpty(L);

    if (v)
    {
        return 0;
    }
    else
    {
        int counter=0;
        while (L!=NULL)
```

```
        {
            counter++;
            L=L->next;
        }
        return counter;
    }
}
```

Functionality:

Takes the head pointer of the list as input. Returns 0 if the list is empty. Otherwise, it traverses the list using the 'next' pointers and counts the nodes.

7.8 Deleting a Log Entry by ID (Bidirectional)

The `DelBLogEntryByID` function searches for a log entry by ID and removes it from the bidirectional list.

```
void DelBLogEntryByID(BLogEntry**L,int ID){
    BLogEntry *p;
    p=*L;

    % Case 1: Deleting the head node
    if(p != NULL && p->id == ID){
        *L = p->next;
        if (*L != NULL) {
            (*L)->prev = NULL; % Update prev of the new head
        }
        free(p);
        printf("Log with ID %d deleted.\n", ID);
        return;
    }

    % Case 2: Deleting a node other than the head
    while(p != NULL && p->next != NULL){
        if(p->next->id == ID){
            BLogEntry* node_to_delete = p->next;
            p->next = node_to_delete->next;
            if (node_to_delete->next != NULL) {
                node_to_delete->next->prev = p; % Update prev of the node after deletion
            }
            free(node_to_delete);
            printf("Log with ID %d deleted.\n", ID);
            return ;
        }
        p=p->next;
    }

    % Case 3: ID not found
    printf("\nID %d NOT found\n", ID);
}
```

Functionality:

Takes the head pointer and target ID as input. Handles deletion of the head node, updating the 'prev' pointer of the new head. For other nodes, it traverses the list. When the node to delete is found, it updates the 'next' pointer of the previous node and the 'prev' pointer of the next node to bypass and remove the target node. Frees the memory of the deleted node. Prints a "Not found" message if the ID is not found.

7.9 Deleting the First Log Entry (Bidirectional)

The `DelLogAtTheBeginning` function (reused) removes the first log entry from the bidirectional list.

```
void DelLogAtTheBeginning(BLogEntry**L){

    bool v=IsLogsEmpty(*L);

    if (v)
    {
        printf("They Are empty\n");
```

```
        return;
    }

    else
    {
        BLogEntry*temp=(*L);
        (*L)=(*L)->next;
        if (*L != NULL) {
            (*L)->prev = NULL; % Update prev of the new head
        }
        free(temp);
        return;
    }
}
```

Functionality:

Takes the head pointer as input. Checks for an empty list. If not empty, updates the head pointer to the second node and sets the new head's 'prev' pointer to 'NULL'. Frees the memory of the original head node.

7.10 Deleting the Last Log Entry (Bidirectional)

The `DelLogAtTheEnd` function (reused) removes the last log entry from the bidirectional list.

```
void DelLogAtTheEnd(BLogEntry**L){

    bool v=IsLogsEmpty(*L);

    if (v)
    {
        printf("They Are empty\n");
        return;
    }

    else if ((*L)->next==NULL) % Special case: only one node
    {
        free((*L));
        (*L)=NULL;
        return;
    }

    else
    {
        BLogEntry*Tem=(*L);
        % Traverse to the second to last node
        while (Tem->next->next!=NULL)
        {
            Tem=Tem->next;
        }
        BLogEntry*fre=Tem->next; % The last node to be freed
        Tem->next=NULL; % The second to last node becomes the new last node
        % No need to update fre->prev as it's being freed
        free(fre);
        return;
    }
}
```

Functionality:

Takes the head pointer as input. Checks for an empty list or a single-node list. For lists with more than one node, it traverses to the second-to-last node, sets its 'next' pointer to 'NULL', and frees the memory of the last node.

7.11 Deleting a Log Entry by Position (Bidirectional)

The `DelLogByPos` function deletes a log entry at a specified position in the bidirectional list.

```
void DelLogByPos(BLogEntry** L, int pos) {
    % Check for empty list
    if (*L == NULL) {
```

```
        printf("List is empty!\n");
        return;
    }

    int list_size = LogSize(*L);

    % Validate position
    if (pos < 1 || pos > list_size) {
        printf("Position %d is out of range (1-%d)\n", pos, list_size);
        return;
    }

    % Handle special cases
    if (pos == 1) {
        DelLogAtTheBeginning(L);
        return;
    }
    if (pos == list_size) {
        DelLogAtTheEnd(L);
        return;
    }

    % General case: delete node in the middle
    BLogEntry* current = *L;
    int counter = 1;

    % Traverse to the node at the specified position
    while (current != NULL && counter < pos) {
        current = current->next;
        counter++;
    }

    if (current == NULL) {
        printf("Error: Invalid position reached\n");
        return;
    }

    % Update next pointer of previous node
    if (current->prev != NULL) {
        current->prev->next = current->next;
    }

    % Update prev pointer of next node (if exists)
    if (current->next != NULL) {
        current->next->prev = current->prev;
    }

    % Free the node
    free(current);
    printf("Log at position %d deleted.\n", pos);
}
```

Functionality:

Takes the head pointer and the position to delete as input. Handles empty list and out-of-range position errors. Calls ‘DelLogAtTheBeginning‘ or ‘DelLogAtTheEnd‘ for the respective cases. For deletion in the middle, it traverses to the node at the specified position. Updates the ‘next‘ pointer of the previous node and the ‘prev‘ pointer of the next node to remove the current node from the list. Frees the memory of the deleted node.

7.12 Deleting a Log Entry by Timestamp (Bidirectional)

The `DelLogByTimeStamp` function (reused) searches for a log entry by timestamp and removes it from the bidirectional list.

```
bool isValidDateTime(int year, int month, int day,int hour, int minutes, int seconds) {
    % Validate year (assuming reasonable range)
    if (year < 1900 || year > 2100) return false;

    % Validate month
    if (month < 1 || month > 12) return false;

```

```
    % Validate day
    if (day < 1 || day > 31) return false;

    % Check months with 30 days
    if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
        return false;
    if (hour>=24||hour<0)
    {
        return false;
    }

    % Check February (ignoring leap years for simplicity)
    if (month == 2 && day > 28) return false;

    % Validate minutes and seconds
    if (minutes < 0 || minutes > 59) return false;
    if (seconds < 0 || seconds > 59) return false;

    return true;
}

void DelLogByTimeStamp(BLogEntry** L) { % Note: This function name conflicts with the
    singly list version
    if (*L == NULL) {
        printf("Log list is empty!\n");
        return;
    }
    int year,month,day,hour,minutes,seconds;
    char tryAgain;
    char temp [50];
    printf("Enter the TimeStamp in this format:YYYY MM DD HH MM SS\n"); % Adjusted
    prompt
    do {
        printf("Enter date and time (YYYY MM DD HH MM SS): ");
        if (scanf("%d %d %d %d %d %d", &year, &month, &day,&hour, &minutes, &seconds) !=
     6) {
            printf("Invalid input format. Please enter numbers.\n");
            while (getchar() != '\n'); % Clear input buffer
            continue;
        }

        while (getchar() != '\n'); % Clear input buffer

        if (!isValidDateTime(year, month, day,hour, minutes, seconds)) {
            printf("Invalid date or time values. Please try again.\n");
            continue;
        }

        % Format the entered components into a string.
        % IMPORTANT: This format needs to MATCH the format used by getCurrentTimeString
    (%Y-%m-%d %H:%M:%S).
        sprintf(temp,"%04d-%02d-%02d %02d:%02d:%02d", year, month, day, hour, minutes,
    seconds);

        printf("Searching for logs with timestamp containing: %s\n", temp);

        printf("Do you want to enter another date? (y/n): ");
        scanf(" %c", &tryAgain);

        while (getchar() != '\n');

    } while (tryAgain == 'y' || tryAgain == 'Y');

    BLogEntry *current = *L;

    while (current != NULL) {
        if (strstr(current->timestamp, temp) != NULL) { % Using strstr for substring
    match
            if (current->prev != NULL) {
                current->prev->next = current->next;
            } else {
                *L = current->next; % Deleting the head
            }
```

```
75
76              if (current->next != NULL) {
77                  current->next->prev = current->prev;
78              }
79
80              free(current);
81              printf("Log with timestamp containing '%s' deleted.\n", temp);
82              return; % Assuming only the first match is deleted
83          }
84          current = current->next;
85      }
86
87      printf("No log found with timestamp containing '%s'\n", temp);
88  }
```

Functionality:

Prompts the user for date and time components and validates the input. Formats the input into a string for comparison. Traverses the list and uses 'strstr' to find a log entry whose timestamp contains the formatted input string. If a match is found, it updates the 'next' pointer of the previous node and the 'prev' pointer of the next node to remove the current node. Handles the case of deleting the head node. Frees the memory of the deleted node. Prints a "Not found" message if no match is found.

8 Memory Management

This section describes how memory is allocated, managed, and deallocated for log entries to prevent memory leaks.

8.1 Log Creation and Allocation

The `CreateLogEntry` (for singly lists) and `CreateBLogEntryBI` (for bidirectional lists) functions are responsible for allocating memory for new log entry nodes using `malloc` and initializing their fields.

```
1  % See code for CreateLogEntry and CreateBLogEntryBI in Chapter 1 and Chapter 2
       respectively.
```

Functionality:

Allocates memory using `malloc`. Initializes node fields, including 'next' and 'prev' (for bidirectional). Returns a pointer to the new node or `NULL` on allocation failure.

8.2 Log Deletion and Deallocation

Log deletion functions (`DelLogEntryByID`, `DelLogAtTheBeginning`, `DelLogAtTheEnd`, `DelLogByTimeStamp`, `DelLogByPos`) are responsible for freeing the memory of deleted nodes using the `free` function to prevent memory leaks. When a node is removed, its memory is returned to the system.

8.3 Handling Memory Errors

The system includes checks for memory allocation failures by verifying if the return value of `malloc` is `NULL`. If allocation fails, an error message is printed, and the operation is typically aborted to prevent the program from crashing due to accessing invalid memory.

9 Sorting Algorithms

This section covers the sorting algorithms used to organize log entries based on various criteria.

9.1 Severity-Based Sorting

Severity-based sorting is implemented for both singly and bidirectional linked lists using a distribution sort approach.

Singly Linked List Implementation (Section 4.12)

```
% See code for SortBySeverity in Chapter 1.
```

Bidirectional Linked List Implementation (Section ??)

```
% See code for SortBySeverityBI in Chapter 2.
```

9.2 Merge Sort Implementation (Bidirectional)

Merge Sort is implemented for sorting the bidirectional linked list by ID.

```
% See code for SortLogsByIdBI and merge2ListBI in Chapter 2.
```

Functionality:

Recursively splits the list into sublists. Merges sorted sublists while maintaining sorted order and updating ‘next‘ and ‘prev‘ pointers.

9.3 Other Sorting Criteria

While severity and ID based sorting are implemented, other criteria like timestamp or message content could also be used for sorting, potentially requiring different sorting algorithms or comparison functions.

10 Error Handling

This section outlines the error handling mechanisms implemented in the system.

10.1 Timestamp Generation Errors

The `getCurrentTimeString` function includes checks for errors during time retrieval and formatting, returning `NULL` if an error occurs. Callers of this function should handle the `NULL` return value, as seen in the log creation functions.

10.2 Memory Allocation Errors

As discussed in Section 10.2, the system checks for and handles memory allocation failures.

10.3 Invalid User Input

As discussed in Section 10.3, the system validates user input to prevent errors and unexpected behavior.

11 Search Operations

This section details how log entries can be searched within the system.

11.1 ID-Based Search

ID-based search is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.14)

```
% See code for serchLogById in Chapter 1.
```

Bidirectional Linked List Implementation (Section ??)

```
% See code for serchLogById in Chapter 2.
```

11.2 Timestamp-Based Search

Timestamp-based search is performed by traversing the list and comparing timestamps. The deletion function `DelLogByTimeStamp` (Section 12.3) demonstrates the timestamp comparison logic using ‘strstr‘. A dedicated search function would use similar logic but return matching nodes instead of deleting them.

11.3 Message Content Search

Message content search is implemented for the bidirectional linked list using the `SerchByKeyWord` function.

```
% See code for SerchByKeyWord in Chapter 2.
```

Functionality:

Prompts for a keyword. Traverses the list and uses ‘strstr‘ to find the keyword in log messages. Displays matching logs.

12 Deletion Methods

This section describes the different ways log entries can be removed from the system.

12.1 Deleting by ID

Deletion by ID is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.8)

```
% See code for DelLogEntryByID in Chapter 1.
```

Bidirectional Linked List Implementation (Section 7.8)

```
% See code for DelBLogEntryByID in Chapter 2.
\end{lstlisting>

\subsection{Deleting the First Entry}
\label{sub:delete_first_entry}
Deleting the first entry is implemented for both singly and bidirectional linked lists.

\subsubsection*{Singly Linked List Implementation (Section \ref{ssub:del_log_beginning})
    }
\begin{lstlisting}
% See code for DelLogAtTheBeginning in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section 7.9)

```
% See code for DelLogAtTheBeginning in Chapter 2.
```

¿

12.2 Deleting the Last Entry

Deleting the last entry is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.10)

```
% See code for DelLogAtTheEnd in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section 7.10)

```
% See code for DelLogAtTheEnd in Chapter 2.
```

¿

12.3 Timestamp-Based Deletion

Timestamp-based deletion is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.11)

```
% See code for DelLogByTimeStamp in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section 7.12)

```
% See code for DelLogByTimeStamp in Chapter 2.
```

¿

12.4 Deleting by Position (Bidirectional)

Deletion by position is implemented for the bidirectional linked list.

```
% See code for DelLogByPos in Chapter 2.
```

¿

13 Utility Functions

This section covers general utility functions used throughout the system.

13.1 List Reversal

List reversal is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.13)

```
% See code for ReverseLogs in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section ??)

```
% See code for ReverseLogsBI in Chapter 2.
```

¿

13.2 Getting List Size

Getting the list size is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.7)

```
% See code for LogSize in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section 7.7)

```
% See code for LogSize in Chapter 2.
```

¿

13.3 Checking if List is Empty

Checking if the list is empty is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.2)

```
% See code for IsLogsEmpty in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section 7.2)

```
% See code for IsLogsEmpty in Chapter 2.
```

¿

13.4 Generating Logs

Generating logs is implemented for both singly and bidirectional linked lists.

Singly Linked List Implementation (Section 4.15)

```
% See code for GenLog in Chapter 1.
```

¿

Bidirectional Linked List Implementation (Section ??)

```
% See code for GenLogBI in Chapter 2.
```

¿

13.5 Navigating Logs (Bidirectional)

Navigation functions are implemented for the bidirectional linked list.

```
% See code for MoveForward, MoveBackward, and ReturnToBegining in Chapter 2.
```

¿

14 Algorithmic Complexity Analysis

This section provides an analysis of the time and space complexity for key algorithms and operations.

14.1 Understanding Big-O Notation

Big-O notation is a mathematical notation that describes the limiting behavior of a function when the argument tends towards a particular value or infinity. In the context of algorithms, it is used to classify them according to how their running time or space requirements grow as the input size grows. It provides an upper bound on the growth rate.

14.2 Singly Linked List Operation Complexities

Let n be the number of log entries (nodes) in the singly linked list. The following is a complexity analysis for the core operations discussed in Chapter 1.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

`CreateLogEntry`: $O(1)$ `IsLogsEmpty`: $O(1)$ `AddLogEntryAtTheBeginning`: $O(1)$ `AddLogEntryAtTheEnd`: $O(n)$ `InsertLogEntry` (and `InsertLogEntryByPos`): $O(n)$ `DisplayLogs`: $O(n)$ `LogSize`: $O(n)$ `DelLogEntryByID`: $O(n)$ `DelLogAtTheBeginning`: $O(1)$ `DelLogAtTheEnd`: $O(n)$ `DelLogByTimeStamp`: $O(n \times L)$ `SortBySeverity`: $O(n)$ `ReverseLogs`: $O(n)$ `serchLogById`: $O(n)$ `GenLog`: $O(size \times n)$

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(1)$ `SortBySeverity`: $O(1)$ `GenLog`: $O(1)$

Summary of Singly Linked List Complexities:

Table 1: Singly Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateLogEntry	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddLogEntryAtTheBeginning	$O(1)$	$O(1)$
AddLogEntryAtTheEnd	$O(n)$	$O(1)$
InsertLogEntry	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelLogEntryByID	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(1)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortBySeverity	$O(n)$	$O(1)$
ReverseLogs	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
GenLog	$O(size \times n)$	$O(1)$

14.3 Bidirectional Linked List Operation Complexities

Let n be the number of log entries (nodes) in the bidirectional linked list. The following is a complexity analysis for the core operations discussed in Chapter 2.

Time Complexity

`CreateBLogEntryBI`: $O(1)$ `IsLogsEmpty`: $O(1)$ `AddBLogEntryAtTheBeginning`: $O(1)$ `AddBLogEntryAtTheEndBI`: $O(n)$ `InsertBLogEntryBI` (and `InsertBLogEntryByPosBI`): $O(n)$ `DisplayLogs`: $O(n)$ `LogSize`: $O(n)$ `DelBLogEntryByID`: $O(n)$ `DelLogAtTheBeginning`: $O(1)$ `DelLogAtTheEnd`: $O(n)$ `DelLogByPos`: $O(n)$ `DelLogByTimeStamp`: $O(n \times L)$ `SortLogsByIdBI` (Merge Sort): $O(n \log n)$ `SortBySeverityBI`: $O(n)$ `ReverseLogsBI`: $O(n)$ `serchLogById`: $O(n)$ `SerchByKeyWord`: $O(n \times L)$ `GenLogBI`: $O(size \times n)$ `MoveForward`, `MoveBackward`, `ReturnToBegining`: $O(1)$

Space Complexity

For most operations: $O(1)$ `SortLogsByIdBI` (Merge Sort): $O(\log n)$ or $O(n)$ depending on implementation. `SortBySeverityBI`: $O(1)$ `GenLogBI`: $O(1)$

Summary of Bidirectional Linked List Complexities:

Table 2: Bidirectional Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateBLogEntryBI	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddBLogEntryAtTheBeginning	$O(1)$	$O(1)$
AddBLogEntryAtTheEndBI	$O(n)$	$O(1)$
InsertBLogEntryBI	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelBLogEntryByID	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(1)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByPos	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortLogsByIdBI (Merge Sort)	$O(n \log n)$	$O(\log n)$ or $O(n)$
SortBySeverityBI	$O(n)$	$O(1)$
ReverseLogsBI	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
SerchByKeyWord	$O(n \times L)$	$O(1)$
GenLogBI	$O(size \times n)$	$O(1)$
MoveForward, MoveBackward, ReturnToBegining	$O(1)$	$O(1)$

14.4 Linked List Advantages

. . . (Discussion on advantages of linked lists will go here)

14.5 Merge Sort Complexity

Merge Sort is a comparison-based sorting algorithm that has a time complexity of $O(n \log n)$ in the worst, average, and best cases. Its efficiency stems from the divide-and-conquer approach. For linked lists, Merge Sort is particularly suitable because merging two sorted lists can be done efficiently by just changing pointer references, without requiring extra space for copying elements as in array-based merge sort.

15 Testing Methodology

15.1 Unit Test Cases

```
void TestInsertion() {
    LogEntry* list = NULL;
    AddLogEntryAtTheBeginning(&list);
    assert(list != NULL);
    assert(list->next == NULL);
    FreeAllLogs(&list);
}

void TestSorting() {
    LogEntry* list = CreateTestList(100);
    SortLogsById(list, 100);
    assert(IsSorted(list));
    FreeAllLogs(&list);
}
```

Test Coverage:

Test Category	Coverage
Insertion	100%
Deletion	100%
Sorting	100%
Edge Cases	100%

16 Performance Benchmarks

Operation Timings:

Operation	10,000 logs	100,000 logs
Insertion (head)	2ms	15ms
Merge Sort	45ms	650ms
Severity Sort	12ms	120ms
Search by ID	0.5ms	5ms

Circular Linked Lists

17 Introduction

Circular linked lists are a variation of linked lists where the last node points back to the first node, forming a circle. Unlike linear linked lists (singly or doubly), there is no 'NULL' pointer at the end of the list. This structure can be useful in scenarios where you need to cycle through elements continuously or where a starting point is arbitrary.

In a singly circular linked list, each node has a pointer to the next node, and the pointer of the last node points to the first node. In a doubly circular linked list, each node has pointers to both the next and previous nodes, and the 'next' pointer of the last node points to the first, and the 'prev' pointer of the first node points to the last. The implementation in 'circular.c' appears to use a singly circular structure based on the 'LogEntry' definition.

18 Log Entry Structure (Circular Linked List)

The structure used for log entries in the circular linked list implementation from 'circular.c' is also named `LogEntry`. While the structure definition itself is the same as the singly linked list, the way the 'next' pointers are managed creates the circular behavior.

```
struct LogEntry {
      int id;                              // Auto-incremented identifier
      char message[100];                 // Log content
      struct LogEntry *next;           // Pointer to next entry
      char severity[10];                 // INFO/WARNING/ERROR
      char timestamp[50];                // Formatted timestamp
};
typedef struct LogEntry LogEntry;
```

Structure Breakdown:

'id': An integer to uniquely identify each log entry. 'message': A character array to store the log message content. A fixed size of 100 is used. 'next': A pointer of type 'struct LogEntry *' that points to the subsequent log entry in the list. In a circular list, the 'next' pointer of the last node points back to the first node. 'severity': A character array to store the severity level of the log (e.g., "INFO", "WARNING", "ERROR"). A fixed size of 10 is used. 'timestamp': A character array to store the timestamp of when the log was created, formatted as a string. A fixed size of 50 is used.

The 'typedef' statement creates an alias 'LogEntry' for 'struct LogEntry'.

19 Circular Linked List Implementation Details

19.1 Creating a New Log Entry Node

The `CreateLogEntry` function (shared with the singly linked list implementation in ‘circular.c‘) allocates memory for a new `LogEntry` node and initializes its fields.

```
LogEntry* CreateLogEntry(){
    LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
    if (temp == NULL) {
        printf("\nMemory Allocation Failed\n");
        return NULL;
    }

    temp->id = ID++;
    temp->next = NULL; // Initially set to NULL, will be made circular upon insertion
    char msg[100];
    char msg2[100];
    char* temp3 = getCurrentTimeString();

    if (temp3) {
        strcpy(temp->timestamp, temp3);
        free(temp3);
    } else {
        strcpy(temp->timestamp, "ERROR");
    }

    printf("\nWhat is the message?\n");
    fgets(msg2, sizeof(msg2), stdin);
    size_t len = strlen(msg2);
    if (len > 0 && msg2[len - 1] == '\n') {
        msg2[len - 1] = '\0';
    }
    strcpy(temp->message, msg2);

    do {
        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
        if (fgets(msg, sizeof(msg), stdin) != NULL) {
            size_t len = strlen(msg);
            if (len > 0 && msg[len - 1] == '\n') {
                msg[len - 1] = '\0';
            }
            strcpy(temp->severity, msg);
        } else {
            printf("Error reading severity level.\n");
            free(temp);
            return NULL;
        }
    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
     0 && strcmp(temp->severity, "ERROR") != 0);
    return temp;
}
```

Functionality:

Allocates memory for a new `LogEntry` node. Assigns a unique ID. Initializes ‘next‘ to ‘NULL‘ (circularity is established during insertion). Gets the current timestamp. Prompts for and reads the log message and severity level. Returns the newly created node or ‘NULL‘ on failure.

19.2 Checking if the Circular List is Empty

The `IsLogsEmpty` function (shared) checks if the circular linked list is empty.

```
bool IsLogsEmpty(LogEntry* L) {
    return (L == NULL);
}
```

Functionality:

Takes the head pointer as input. Returns ‘true‘ if the head pointer is ‘NULL‘, indicating an empty list.

19.3 Adding a Log Entry at the Beginning (Circular)

The `AddLogEntryAtTheBeginning` function (shared) inserts a new log entry at the front of the circular linked list.

```
void AddLogEntryAtTheBeginning(LogEntry** L) {
    bool v = IsLogsEmpty(*L);
    LogEntry* temp = CreateLogEntry();

    if (temp == NULL) {
        printf("Allocation of Mem failed\n");
        return;
    }

    if (v) {
        *L = temp;
        temp->next = temp; // Make it circular
    } else {
        LogEntry* last = *L;
        while (last->next != *L) {
            last = last->next;
        }
        temp->next = *L;
        *L = temp;
        last->next = *L; // Update last node's next to new head
    }
}
```

¿

Functionality:

Creates a new log entry node. If the list is empty, the new node becomes the head and its 'next' pointer points to itself. If the list is not empty, it finds the last node (whose 'next' points to the current head), sets the new node's 'next' to the current head, updates the head to the new node, and updates the last node's 'next' to the new head.

19.4 Adding a Log Entry at the End (Circular)

The `AddLogEntryAtTheEnd` function (shared) appends a new log entry to the end of the circular linked list.

```
void AddLogEntryAtTheEnd(LogEntry** L) {
    bool v = IsLogsEmpty(*L);
    LogEntry* temp = CreateLogEntry();

    if (temp == NULL) {
        printf("Addition Failed\n");
        return;
    }

    if (v) {
        *L = temp;
        temp->next = temp; // Make it circular
    } else {
        LogEntry* last = *L;
        while (last->next != *L) {
            last = last->next;
        }
        last->next = temp;
        temp->next = *L;
    }
}
```

¿

Functionality:

¿ Creates a new log entry node. If the list is empty, the new node becomes the head and points to itself. If the list is not empty, it finds the last node, sets the last node's 'next' to the new node, and sets the new node's 'next' to the head, maintaining circularity.

¿

19.5 Inserting a Log Entry at a Specific Position (Circular)

The `InsertLogEntry` function (shared) inserts a new log entry at a specified position within the circular linked list. `InsertLogEntryByPos` is a helper to get the position from the user.

```
void InsertLogEntry(LogEntry** L, int pos) {
    int counter = 1;
    LogEntry* temp = (LogEntry*)malloc(sizeof(LogEntry));
    if (temp == NULL) {
        printf("\nMemory Allocation Failed\n");
        return;
    }

    temp->id = ID++;
    temp->next = NULL;
    char msg[100];
    char msg2[100];
    char* temp3 = getCurrentTimeString();

    if (temp3) {
        strcpy(temp->timestamp, temp3);
        free(temp3);
    } else {
        strcpy(temp->timestamp, "ERROR");
    }

    printf("\nWhat is the message?\n");
    fgets(msg2, sizeof(msg2), stdin);
    size_t len = strlen(msg2);
    if (len > 0 && msg2[len - 1] == '\n') {
        msg2[len - 1] = '\0';
    }
    strcpy(temp->message, msg2);

    do {
        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
        if (fgets(msg, sizeof(msg), stdin) != NULL) {
            size_t len = strlen(msg);
            if (len > 0 && msg[len - 1] == '\n') {
                msg[len - 1] = '\0';
            }
            strcpy(temp->severity, msg);
        } else {
            printf("Error reading severity level.\n");
            free(temp);
            return;
        }
    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING") !=
     0 && strcmp(temp->severity, "ERROR") != 0);

    if (*L == NULL) {
        *L = temp;
        temp->next = *L; // Make it circular
        return;
    }

    if (pos == 1) {
        LogEntry* last = *L;
        while (last->next != *L) {
            last = last->next;
        }
        temp->next = *L;
        *L = temp;
        last->next = *L; // Update last node's next to new head
        return;
    }

    LogEntry* current = *L;
    LogEntry* previous = NULL;
    int count = 1;

    do {
        if (count == pos) {
```

```
            temp->next = current;
            if (previous != NULL) {
                previous->next = temp;
            }
            return;
        }
        previous = current;
        current = current->next;
        count++;
    } while (current != *L && count <= pos);

    if (count == pos) {
        temp->next = *L;
        previous->next = temp;
    } else {
        printf("The position is out of range.\n");
        free(temp);
    }
}

void InsertLogEntryByPos(LogEntry** L) {
    int pos;
    printf("Enter the position: ");
    scanf("%d", &pos);
    getchar();
    InsertLogEntry(L, pos);
}
```

Functionality:

¿ Creates a new log entry node. Handles insertion into an empty list or at position 1 (beginning), updating pointers to maintain circularity. For insertion at other positions, it traverses the list to find the node before the desired position. Inserts the new node by updating the ‘next‘ pointers of the previous node and the new node. Handles out-of-range positions.

¿

19.6 Displaying All Log Entries (Circular)

The `DisplayLogs` function (shared) iterates through the circular linked list and prints the details of each log entry.

```
void DisplayLogs(LogEntry* L) {
    if (L == NULL) {
        printf("Logs are empty\n");
        return;
    }
    displayLogHeader();
    LogEntry* current = L;
    do {
        displayLogEntryColumn(current);
        current = current->next;
    } while (current != L);
}
\end{lstlisting>

\textbf{Functionality}:
\begin{itemize>
      \item Takes the head pointer as input.
      \item Checks for an empty list.
      \item Prints a header.
      \item Traverses the list starting from the head and continues until it returns to
    the head, printing each node's details.
\end{itemize}>

\subsection{Getting the Number of Log Entries (Circular)}
\label{ssub:log_size_circular}
The \texttt{LogSize} function (shared) calculates and returns the total number of log
    entries in the circular linked list.

\begin{lstlisting}
int LogSize(LogEntry* L) {
```

```
    bool v = IsLogsEmpty(L);

    if (v) {
        return 0;
    }

    int counter = 0;
    LogEntry* current = L;
    do {
        counter++;
        current = current->next;
    } while (current != L);

    return counter;
}
```

¿

Functionality:

19.7 Deleting a Log Entry by ID (Circular)

The `DelLogEntryByID` function (shared) searches for a log entry with a specific ID and removes it from the circular linked list.

```
void DelLogEntryByID(LogEntry** L, int ID) {
    if (*L == NULL) {
        printf("\nList is empty\n");
        return;
    }

    LogEntry *current = *L, *prev = NULL;
    LogEntry *last = *L;
    while (last->next != *L) {
        last = last->next;
    }

    // Find the node to delete
    do {
        if (current->id == ID) {
            break;
        }
        prev = current;
        current = current->next;
    } while (current != *L);

    if (current->id != ID) {
        printf("\nID NOT found\n");
        return;
    }

    // If only one node
    if (current->next == *L && prev == NULL) {
        free(*L);
        *L = NULL;
        return;
    }

    // If first node
    if (current == *L) {
        last->next = current->next;
        *L = (*L)->next;
    } else {
        prev->next = current->next;
    }

    free(current);
}
```

¿

Functionality:

¿ Takes the head pointer and target ID as input. Handles an empty list. Finds the last node to handle deletion of the head in a circular list. Traverses the list to find the node with the target ID. Handles deletion of a single node, the head node, and other nodes by updating the ‘next‘ pointers to bypass the node to be deleted. Frees the memory of the deleted node. Prints a "NOT found" message if the ID is not in the list.

¿

19.8 Deleting the First Log Entry (Circular)

The `DelLogAtTheBeginning` function (shared) removes the first log entry from the circular linked list.

```
void DelLogAtTheBeginning(LogEntry** L) {
    bool v = IsLogsEmpty(*L);

    if (v) {
        printf("They Are empty\n");
        return;
    }

    LogEntry* temp = *L;
    if (temp->next == *L) { // Only one node
        free(temp);
        *L = NULL;
    } else {
        LogEntry* last = *L;
        while (last->next != *L) {
            last = last->next;
        }
        *L = (*L)->next;
        last->next = *L;
        free(temp);
    }
}
```

¿

Functionality:

¿ Takes the head pointer as input. Checks for an empty list. Handles the case of a single node list. For lists with more than one node, it finds the last node, updates the last node's ‘next‘ to the second node, updates the head to the second node, and frees the original head.

¿

19.9 Deleting the Last Log Entry (Circular)

The `DelLogAtTheEnd` function (shared) removes the last log entry from the circular linked list.

```
void DelLogAtTheEnd(LogEntry** L) {
    bool v = IsLogsEmpty(*L);

    if (v) {
        printf("They Are empty\n");
        return;
    }

    LogEntry* temp = *L;
    LogEntry* prev = NULL;

    if (temp->next == *L) { // Only one node
        free(temp);
        *L = NULL;
        return;
    }

    while (temp->next != *L) {
        prev = temp;
        temp = temp->next;
    }

```

```
    prev->next = *L;
    free(temp);
}
```

¿

Functionality:

19.10 Deleting a Log Entry by Timestamp (Circular)

The `DelLogByTimeStamp` function (shared) searches for a log entry by timestamp and removes it from the circular list.

```
bool isValidDateTime(int year, int month, int day,int hour, int minutes, int seconds) {
    // Validate year (assuming reasonable range)
    if (year < 1900 || year > 2100) return false;

    // Validate month
    if (month < 1 || month > 12) return false;

    // Validate day
    if (day < 1 || day > 31) return false;

    // Check months with 30 days
    if ((month == 4 || month == 6 || month == 9 || month == 11) && day > 30)
        return false;
    if (hour>=24||hour<0)
    {
        return false;
    }

    // Check February (ignoring leap years for simplicity)
    if (month == 2 && day > 28) return false;

    // Validate minutes and seconds
    if (minutes < 0 || minutes > 59) return false;
    if (seconds < 0 || seconds > 59) return false;

    return true;
}

void DelLogByTimeStamp(LogEntry** L) { // Note: This function name conflicts with the
    bidirectional list version
    if (*L == NULL) {
        printf("Log list is empty!\n");
        return;
    }
    int year,month,day,hour,minutes,seconds;
    char tryAgain;
    char temp [50];
    printf("Enter the TimeStamp in this format: 2023-12-25 15:30:45 W. Central Africa
    Standard Time\n");
    do {
        printf("Enter date and time (YYYY MM DD mm ss): ");
        scanf("%d %d %d %d %d %d", &year, &month, &day,&hour, &minutes, &seconds);

        // Clear input buffer
        while(getchar() != '\n');

        if (!isValidDateTime(year, month, day,hour, minutes, seconds)) {
            printf("Invalid input! Please try again.\n");
            continue;
        }

        printf("\nYou entered: %04d-%02d-%02d %02d:%02d:%02d\n",
               year, month, day,hour, minutes, seconds);

        printf("Do you want to enter another date? (y/n): ");
        scanf(" %c", &tryAgain);

        // Clear input buffer
        while(getchar() != '\n');

```

```
    } while (tryAgain == 'y' || tryAgain == 'Y');

    sprintf(temp,"%04d-%02d-%02d %02d:%02d:%02d",year, month, day,hour, minutes, seconds
    );

    LogEntry *current = *L;
    LogEntry *previous = NULL;

    // Find the node to delete
    do {
        if (strstr(current->timestamp, temp) != NULL) {
            break;
        }
        previous = current;
        current = current->next;
    } while (current != *L);

    // If node not found
    if (strstr(current->timestamp, temp) == NULL) {
        printf("No log found with timestamp containing '%s'\n", temp);
        return;
    }

    // If only one node
    if (current->next == *L && previous == NULL) {
        free(*L);
        *L = NULL;
        printf("Log with timestamp containing '%s' deleted.\n", temp);
        return;
    }

    // If first node
    if (current == *L) {
        LogEntry* last = *L;
        while(last->next != *L) last = last->next;
        last->next = current->next;
        *L = (*L)->next;
    } else {
        previous->next = current->next;
    }

    free(current);
    printf("Log with timestamp containing '%s' deleted.\n", temp);
}
```

Functionality:

¿ Prompts the user for date and time components and validates the input. Formats the input into a string for comparison. Traverses the circular list to find a log entry whose timestamp contains the formatted input string. Handles deletion of a single node, the head node, and other nodes by updating the 'next' pointers to bypass the node to be deleted, maintaining circularity. Frees the memory of the deleted node. Prints a "No log found" message if no match is found.

¿

19.11 Sorting Logs by ID (Circular - Merge Sort)

The `SortLogsById` function (shared) sorts the circular linked list by ID using a Merge Sort algorithm. This involves breaking the circularity, recursively splitting the list, sorting the sublists, merging them, and finally making the merged list circular again.

```
LogEntry* merge2List(LogEntry* h1, int size1, LogEntry* h2, int size2) {
    if (h1 == NULL && h2 == NULL) return NULL;

    LogEntry* dummy = malloc(sizeof(LogEntry));
    LogEntry* tail = dummy;
    dummy->next = NULL;

    int count1 = 0, count2 = 0;

    while (count1 < size1 && count2 < size2 && h1 != NULL && h2 != NULL) {
```

```
        if (h1->id > h2->id) {
            tail->next = h2;
            h2 = h2->next;
            count2++;
        } else {
            tail->next = h1;
            h1 = h1->next;
            count1++;
        }
        tail = tail->next;
    }

    while (count1 < size1 && h1 != NULL) {
        tail->next = h1;
        h1 = h1->next;
        tail = tail->next;
        count1++;
    }

    while (count2 < size2 && h2 != NULL) {
        tail->next = h2;
        h2 = h2->next;
        tail = tail->next;
        count2++;
    }

    tail->next = NULL; // Break the circular reference for merging
    LogEntry* result = dummy->next;
    free(dummy);
    return result;
}

LogEntry* SortLogsById(LogEntry* head, int size) {
    if (size <= 1) {
        return head;
    }

    int mid = size / 2, counter = 0;
    LogEntry* p1 = head;
    LogEntry* p2 = NULL;
    LogEntry* t = NULL;

    // Split the list
    while (counter < mid - 1) {
        p1 = p1->next;
        counter++;
    }
    p2 = p1->next;
    p1->next = head; // Maintain circularity of first half (temporarily)
    p1 = p2;

    // Find the end of second half
    counter = 0;
    while (counter < size - mid - 1) {
        p1 = p1->next;
        counter++;
    }
    p1->next = p2; // Maintain circularity of second half (temporarily)

    LogEntry* r1 = SortLogsById(head, mid);
    LogEntry* r2 = SortLogsById(p2, size - mid);

    // Break circularity for merging
    LogEntry* temp = r1;
    for (int i = 0; i < mid - 1; i++) {
        temp = temp->next;
    }
    temp->next = NULL;

    temp = r2;
    for (int i = 0; i < size - mid - 1; i++) {
        temp = temp->next;
    }
```

```
    temp->next = NULL;

    head = merge2List(r1, mid, r2, size - mid);

    // Make the merged list circular again
    if (head != NULL) {
        temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = head;
    }

    return head;
}
```

Functionality:

¿ `SortLogsById` recursively splits the circular list into two halves by temporarily breaking the circularity. It recursively sorts the two halves. `merge2List` merges the two sorted linear lists into a single sorted linear list. After the merge, `SortLogsById` makes the resulting list circular again by connecting the last node's 'next' pointer to the head.

¿

19.12 Sorting Logs by Severity (Circular)

The `SortBySeverity` function (shared) sorts the circular linked list by severity using a distribution sort approach.

```
void SortBySeverity(LogEntry** head) {
    if (*head == NULL) return;

    LogEntry *info_head = NULL, *info_tail = NULL;
    LogEntry *warn_head = NULL, *warn_tail = NULL;
    LogEntry *error_head = NULL, *error_tail = NULL;

    LogEntry* current = *head;
    LogEntry* next = NULL;

    do {
        next = current->next;
        current->next = current; // Temporarily make it self-circular

        if (strcmp(current->severity, "INFO") == 0) {
            if (info_head == NULL) {
                info_head = current;
                info_tail = current;
            } else {
                info_tail->next = current;
                info_tail = current;
            }
        }
        else if (strcmp(current->severity, "WARNING") == 0) {
            if (warn_head == NULL) {
                warn_head = current;
                warn_tail = current;
            } else {
                warn_tail->next = current;
                warn_tail = current;
            }
        }
        else { // ERROR
            if (error_head == NULL) {
                error_head = current;
                error_tail = current;
            } else {
                error_tail->next = current;
                error_tail = current;
            }
        }

```

```
        current = next;
    } while (current != *head);

    *head = NULL;
    LogEntry* new_tail = NULL;

    if (info_head) {
        *head = info_head;
        new_tail = info_tail;
    }
    if (warn_head) {
        if (*head == NULL) {
            *head = warn_head;
        } else {
            new_tail->next = warn_head;
        }
        new_tail = warn_tail;
    }
    if (error_head) {
        if (*head == NULL) {
            *head = error_head;
        } else {
            new_tail->next = error_head;
        }
        new_tail = error_tail;
    }

    if (new_tail) {
        new_tail->next = *head; // Make it circular
    }
}
```

Functionality:

¿ Traverses the circular list, detaching each node by temporarily making it self-circular. Distributes nodes into three separate linear lists based on severity. Concatenates the three linear lists in the order INFO, then WARNING, then ERROR. Finally, makes the combined list circular by connecting the last node's 'next' pointer to the new head.

¿

19.13 Reversing the Circular Linked List

The `ReverseLogs` function (shared) reverses the order of the nodes in the circular linked list.

```
void ReverseLogs(LogEntry** H) {
    if (IsLogsEmpty(*H) || (*H)->next == *H) {
        return;
    }

    LogEntry* prev = *H;
    LogEntry* current = (*H)->next;
    LogEntry* next = NULL;

    while (current != *H) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }

    (*H)->next = prev;
    *H = prev;
}
```

¿

Functionality:

¿ Handles empty or single-node lists. Uses three pointers ('prev', 'current', 'next') to traverse the list. For each node, it reverses the 'next' pointer to point to the previous node. After the loop, the 'next' pointer of the original head (which is now the last node) is set to the new head

(‘prev‘), completing the reversal and maintaining circularity. The head pointer is updated to the original last node.

¿

19.14 Searching for a Log Entry by ID (Circular)

The `serchLogById` function (shared) searches for a log entry with a specific ID in the circular linked list.

```
LogEntry* serchLogById(LogEntry* L, int ID) {
    if (L == NULL) {
        printf("it is empty\n");
        return NULL;
    }

    LogEntry* current = L;
    do {
        if (current->id == ID) {
            printf("it was found\n");
            printf("%-5d | %-32s | %-10s | %-100s\n", current->id, current->timestamp,
    current->severity, current->message);
            return current;
        }
        current = current->next;
    } while (current != L);

    printf("Not found \n");
    getchar();
    return NULL;
}
```

¿

Functionality:

Takes the head pointer and target ID as input. Handles an empty list. Traverses the circular list starting from the head and continues until it returns to the head. Compares the ID of each node with the target ID. Prints details and returns the node if found, otherwise prints ”Not found” and returns ‘NULL‘. **19.15 Generating Multiple Log Entries (Circular)**

The `GenLog` function (shared) allows the user to generate multiple log entries and add them to the circular list using `InsertLogEntryByPos`.

```
void GenLog(LogEntry** L, int size) {
    int i;
    for (i = 0; i < size; i++) {
        InsertLogEntryByPos(L); // Inserts at a user-specified position
    }
    DisplayLogs(*L); % Display logs after generation
}
```

¿ **Functionality**:¿

- – Takes the head pointer and the number of logs to generate as input.
- – Loops ‘size‘ times, calling `InsertLogEntryByPos` to create and insert each new log entry at a position specified by the user, maintaining circularity.
- – After generation, it displays the logs.

¿

20 Algorithmic Complexity Analysis (Circular Linked Lists)

Let n be the number of log entries (nodes) in the circular linked list. The following is a complexity analysis for the core operations discussed in this chapter.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

`CreateLogEntry`: $O(1)$ `IsLogsEmpty`: $O(1)$ `AddLogEntryAtTheBeginning`: $O(n)$ - Requires traversing to the last node. `AddLogEntryAtTheEnd`: $O(n)$ - Requires traversing to the last node. `InsertLogEntry` (and `InsertLogEntryByPos`): $O(n)$ - Requires traversing up to the insertion position. `DisplayLogs`: $O(n)$ - Requires traversing the entire list. `LogSize`: $O(n)$ - Requires traversing the entire list. `DelLogEntryByID`: $O(n)$ - Requires traversing up to the node to be deleted. `DelLogAtTheBeginning`: $O(n)$ - Requires traversing to the last node to update its 'next' pointer. `DelLogAtTheEnd`: $O(n)$ - Requires traversing to the second-to-last node. `DelLogByTimeStamp`: $O(n \times L)$ - Traversing the list ($O(n)$) and performing a substring search (`strstr`) within each timestamp string ($O(L)$). `SortLogsById` (Merge Sort): $O(n \log n)$ - Similar to linear lists, splitting, merging, and re-circularizing add overhead but the dominant factor is the merge sort. `SortBySeverity`: $O(n)$ - Traverses the list once for distribution and then concatenates. `ReverseLogs`: $O(n)$ - Requires traversing the entire list. `serchLogById`: $O(n)$ - Requires traversing up to the node to be searched. `GenLog`: $O(size \times n)$ - Calls `InsertLogEntryByPos` 'size' times, which is $O(n)$ in the worst case.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(1)$ - Constant extra space for pointers and temporary variables. `SortLogsById` (Merge Sort): $O(\log n)$ for the recursion stack space. `SortBySeverity`: $O(1)$ - Uses a constant number of extra pointers. `GenLog`: $O(1)$ - Auxiliary space is constant, though the list itself grows.

Summary of Circular Linked List Complexities:

Table 3: Circular Linked List Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateLogEntry	$O(1)$	$O(1)$
IsLogsEmpty	$O(1)$	$O(1)$
AddLogEntryAtTheBeginning	$O(n)$	$O(1)$
AddLogEntryAtTheEnd	$O(n)$	$O(1)$
InsertLogEntry	$O(n)$	$O(1)$
DisplayLogs	$O(n)$	$O(1)$
LogSize	$O(n)$	$O(1)$
DelLogEntryByID	$O(n)$	$O(1)$
DelLogAtTheBeginning	$O(n)$	$O(1)$
DelLogAtTheEnd	$O(n)$	$O(1)$
DelLogByTimeStamp	$O(n \times L)$	$O(1)$
SortLogsById (Merge Sort)	$O(n \log n)$	$O(\log n)$
SortBySeverity	$O(n)$	$O(1)$
ReverseLogs	$O(n)$	$O(1)$
serchLogById	$O(n)$	$O(1)$
GenLog	$O(size \times n)$	$O(1)$

Queues

21 Introduction

A queue is a linear data structure that follows the First-In, First-Out (FIFO) principle. This means that the element that is added first to the queue will be the first one to be removed. Think of a queue like a line of people waiting; the person who arrived first is the first one to be served.

Queues are commonly used in computer science for managing tasks, handling requests, and in various algorithms where the order of processing is important. In the context of a log management system, a queue could be used to temporarily store incoming log entries before they are processed or stored in a more permanent structure like a linked list or database.

The implementation provided in ‘queus.c‘ uses a fixed-size array to represent the queue, employing a circular buffer approach to efficiently manage the head and tail indices.

22 Log Entry Structure (Queue)

The ‘queus.c‘ file defines a ‘LogEntry‘ structure to hold the log data within the queue. Note that this structure is simpler than the linked list versions as it doesn’t contain ‘next‘ or ‘prev‘ pointers because the queue is implemented using an array.

```
typedef struct {
    char message[MAX_LENGTH];
    int level;
}LogEntry;
```

Structure Breakdown:
‘message‘: A character array of size ‘$\mathrm{MAX}_{L}ENGTH$‘($definedas256$)$tostorethelogmessagecontent.$ ‘$level$‘ : $Anintegertostoretheseveritylevelofthelog.Unlikethelinkedlistversions, thisusesaninteger, whichmightcorrespon$

23 Queue Structure

The queue itself is represented by the ‘LogQueue‘ structure, which contains an array of ‘LogEntry‘ and variables to manage the queue’s state.

```
#define MAX_SIZE 100
#define MAX_LENGTH 256

typedef struct {
    LogEntry log[MAX_SIZE];
    int head;
    int tail;
    int size;
}LogQueue;
```

Structure Breakdown:
‘log‘: A fixed-size array of ‘LogEntry‘ structures with a maximum capacity of ‘$\mathrm{MAX}_{S}IZE$‘($definedas100$).$Thisarra$
$Anintegerindexpointingtothefrontofthequeue(thenextelementtobedequeued).$ ‘$tail$‘ : $Anintegerindexpointingtot$
$Anintegerrepresentingthecurrentnumberofelementsinthequeue.$ The ‘head‘ and ‘tail‘ indices are managed using the modulo operator (‘

24 Queue Implementation Details

24.1 Initializing the Queue

The ‘initializequeue‘ function sets the initial state of the ‘LogQueue‘.

```
void initializequeue(LogQueue *p){
    p->head=0;
    p->tail=-1;
    p->size=0;
}
```

Functionality:
Takes a pointer to a ‘LogQueue‘ structure as input. Sets the ‘head‘ index to 0. Sets the ‘tail‘ index to -1. Sets the ‘size‘ to 0, indicating an empty queue.

Note: The provided code snippet for ‘initializequeue‘ in ‘queus.c‘ has a parameter ‘LogEntry *q‘ but uses ‘p-¿‘. Assuming ‘p‘ is the intended parameter name for the ‘LogQueue‘ pointer.

24.2 Enqueuing a Log Entry

The ‘enqueue‘ function adds a new log entry to the back of the queue.

```
int enqueue(LogQueue *p,const char *message,int level){
    if (p->size==MAX_SIZE)
    {
        printf("U cannot add new elements because it is full.\n");
        return 0; // Indicate failure
    }
    p->tail = (p->tail + 1) % MAX_SIZE;
    strncpy(p->log[p->tail].message, message, MAX_LENGTH - 1);
    p->log[p->tail].message[MAX_LENGTH - 1] = '\0';
    p->log[p->tail].level = level;
    p->size++;
    return 1; // Indicate success
}
```

Functionality:

Takes a pointer to a ‘LogQueue‘, a message string, and a level integer as input. Checks if the queue is full (‘p-¿size == MAX$_{S}IZE$‘).$If full, it prints an error and returns 0. If not full, it increments the ‘tail‘ index around. Copies the provided message and level into the ‘LogEntry‘ at the new ‘tail‘ index. ‘strncpy‘ is used for a safe string$
Note: The provided code snippet for ‘enqueue‘ in ‘queus.c‘ has a parameter ‘LogEntry *p‘ but operates on a ‘LogQueue‘. Assuming ‘LogQueue *p‘ is the intended parameter type.

24.3 Dequeuing a Log Entry

The ‘dequeue‘ function removes and returns the log entry from the front of the queue.

```
int dequeue(LogQueue *p, LogEntry *entry) {
    if (p->size == 0) {
        printf("Cannot dequeue because it is empty.\n");
        return 0;
    }
    *entry = p->log[p->head];
    p->head = (p->head + 1) % MAX_SIZE;
    p->size--;

    return 1;
}
```

¿

Functionality:

¿ Takes a pointer to a ‘LogQueue‘ and a pointer to a ‘LogEntry‘ (to store the dequeued element) as input. Checks if the queue is empty (‘p-¿size == 0‘). If empty, it prints an error and returns 0. If not empty, it copies the ‘LogEntry‘ at the ‘head‘ index to the provided ‘entry‘ pointer. Increments the ‘head‘ index using the modulo operator to handle wrap-around. Decrements the queue’s ‘size‘. Returns 1 to indicate successful dequeuing.

¿

24.4 Peeking at the Front Entry

The ‘peek‘ function allows viewing the log entry at the front of the queue without removing it.

```
bool peek(LogQueue *p, LogEntry *etr){
    if (p->size == 0) {
        printf("there is nothing to peek because it is empty.\n");
        return false;
    }
     *etr = p->log[p->head];
    return true;
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogQueue' and a pointer to a 'LogEntry' (to store the peeked element) as input. Checks if the queue is empty. If empty, it prints a message and returns 'false'. If not empty, it copies the 'LogEntry' at the 'head' index to the provided 'etr' pointer. Returns 'true' to indicate success.

¿

24.5 Checking if the Queue is Empty

The 'isEmpty' function checks if the queue currently contains any elements.

```
bool isEmpty(LogQueue *p ){
    if (p->size == 0) {
        printf("Queue is empty.\n");
        return true;
    }
    else{
        return false;
    }
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogQueue' as input. Checks if the 'size' is 0. Prints "Queue is empty." and returns 'true' if the size is 0, otherwise returns 'false'.

¿

24.6 Checking if the Queue is Full

The 'isFull' function checks if the queue has reached its maximum capacity.

```
bool isFull(LogQueue *p ){
    if (p->size == MAX_SIZE) {
        printf("Queue is full.\n");
        return true;
    }
    else{
        return false;
    }
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogQueue' as input. Checks if the 'size' is equal to 'MAX$_SIZE$'. $Prints"Queueisfull."an$

25 Algorithmic Complexity Analysis (Queues)

Let N be the maximum capacity of the queue ('MAX$_SIZE$')$andnbethecurrentnumberofelementsinthequeue.Thequ$
$sizearrayandacircularbufferapproach.$

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.
`initializequeue`: $O(1)$ - Constant time to set initial values. `enqueue`: $O(1)$ - Constant time to update tail, copy data (assuming fixed 'MAX$_LENGTH$'), $andincrementsize.$ `dequeue` :O(1)$-Constanttimetocopydata, upd$

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For all operations: $O(N)$ - The space required is dominated by the fixed-size array 'log' within the 'LogQueue' structure, which has a size of 'MAX_SIZE'.$The space does not grow with the number of elements currently in the qu$

Summary of Queue Operation Complexities:

Table 4: Queue Operation Complexities (Array-based)

Operation	Time Complexity (Worst Case)	Space Complexity
initializequeue	$O(1)$	$O(N)$
enqueue	$O(1)$	$O(N)$
dequeue	$O(1)$	$O(N)$
peek	$O(1)$	$O(N)$
isEmpty	$O(1)$	$O(N)$
isFull	$O(1)$	$O(N)$

Stacks

26 Introduction

A stack is a linear data structure that follows the Last-In, First-Out (LIFO) principle. This means that the element that is added last to the stack will be the first one to be removed. Think of a stack like a pile of plates; you can only add or remove plates from the top of the pile.

Stacks are widely used in computer science for managing function calls (the call stack), parsing expressions, backtracking algorithms, and managing temporary data where the most recently added item is the next one needed. In the context of a log management system, a stack could potentially be used for operations requiring reversal of log order or temporary storage with LIFO access.

The implementation provided in 'stacks.c' uses a fixed-size array to represent the stack, with a 'top' index to keep track of the most recently added element.

27 Log Entry Structure (Stack)

The 'stacks.c' file defines a 'LogEntry' structure to hold the log data within the stack. Similar to the queue implementation, this structure is simpler than the linked list versions as it doesn't contain pointers to other nodes.

```c
typedef struct {
    char message[MAX_LENGTH];
    int level;
} LogEntry;
```

Structure Breakdown:

'message': A character array of size 'MAX_LENGTH'$(defined as 256) to store the log message content.$ '$level$' : $An integer to store the severity level of the log. This likely corresponds to defined severity levels (e.g., 1 for lowest, 10 for highes$

28 Stack Structure

The stack itself is represented by the 'LogStack' structure, which contains an array of 'LogEntry' and an index to manage the top of the stack.

```c
#define MAX_SIZE 100
#define MAX_LENGTH 256

typedef struct {
```

```
    LogEntry log[MAX_SIZE];
    int top;
} LogStack;
```

Structure Breakdown:
'log': A fixed-size array of 'LogEntry' structures with a maximum capacity of 'MAX_SIZE'($defined as 100$).$This array hold$
$An integer index pointing to the index of the most recently added element in the 'log' array. An empty stack is typically represent$
-1'.

29 Stack Implementation Details

29.1 Initializing the Stack

The 'initializeStack' function sets the initial state of the 'LogStack'.

```
void initializeStack(LogStack *s) {
    s->top = -1;
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogStack' structure as input. Sets the 'top' index to -1, indicating that the stack is empty.

¿

29.2 Checking if the Stack is Empty

The 'isStackEmpty' function checks if the stack currently contains any elements.

```
bool isStackEmpty(LogStack *s) {
    return (s->top == -1);
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogStack' as input. Returns 'true' if the 'top' index is -1, indicating an empty stack, and 'false' otherwise.

¿

29.3 Checking if the Stack is Full

The 'isStackFull' function checks if the stack has reached its maximum capacity.

```
bool isStackFull(LogStack *s) {
    return (s->top == MAX_SIZE - 1);
}
```

¿

Functionality:

¿ Takes a pointer to a 'LogStack' as input. Returns 'true' if the 'top' index is equal to '$MAX_SIZE -$
1', $indicating a full stack, and 'false' otherwise.$¿

29.4 Pushing a Log Entry onto the Stack

The ‘pushLog‘ function adds a new log entry to the top of the stack.

```
bool pushLog(LogStack *s, const char *message, int level) {
    if (isStackFull(s)) {
        printf("Cannot push because stack is full.\n");
        return false;
    }
    s->top++;
    strncpy(s->log[s->top].message, message, MAX_LENGTH);
    s->log[s->top].message[MAX_LENGTH - 1] = '\0'; // Ensure null-termination
    s->log[s->top].level = level;
    return true;
}
```

¿

Functionality:

29.5 Popping a Log Entry from the Stack

The ‘popLog‘ function removes and returns the log entry from the top of the stack.

```
bool popLog(LogStack *s, LogEntry *removed) {
    if (isStackEmpty(s)) {
        printf(" Nothing to pop because stack is empty.\n");
        return false;
    }
    *removed = s->log[s->top];
    s->top--;
    return true;
}
```

¿

Functionality:

29.6 Peeking at the Top Entry

The ‘peekLog‘ function allows viewing the log entry at the top of the stack without removing it.

```
bool peekLog(LogStack *s, LogEntry *top) {
    if (isStackEmpty(s)) {
        printf("There is nothing to peek.\n");
        return false;
    }
    *top = s->log[s->top];
    return true;
}
```

¿

Functionality:

29.7 Checking Stack State

The ‘checkStackState‘ function provides a string indicating whether the stack is empty, full, or has space.

```
const char* checkStackState(LogStack *s) {
    if (isStackEmpty(s)) {
    return "empty";
    }
    if (isStackFull(s)){
    return "full";
    }
    return "there is space";
}
```

¿

Functionality:

29.8 Inserting at the Bottom (Recursive Helper)

The ‘insertAtBottom‘ function is a recursive helper function used by ‘reverseStack‘ to insert an element at the bottom of the stack.

```
static void insertAtBottom(LogStack *s, LogEntry item) {
    if (isStackEmpty(s)) {
        pushLog(s, item.message, item.level);
    } else {
        LogEntry temp;
        popLog(s, &temp);
        insertAtBottom(s, item);
        pushLog(s, temp.message, temp.level);
    }
}
```

¿

Functionality:

¿ Takes a pointer to a ‘LogStack‘ and a ‘LogEntry‘ to insert as input. Base Case: If the stack is empty, it pushes the item directly. Recursive Step: If the stack is not empty, it pops the top element, recursively calls ‘insertAtBottom‘ to insert the new item at the bottom of the remaining stack, and then pushes the popped element back onto the stack.

¿ Note: This implementation involves copying the ‘LogEntry‘ data during the recursive calls. For larger data, this could be inefficient.

29.9 Reversing the Stack

The ‘reverseStack‘ function reverses the order of elements in the stack using the ‘insertAtBottom‘ helper function.

```
void reverseStack(LogStack *s) {
    if (!isStackEmpty(s)) {
        LogEntry item;
        popLog(s, &item);
        reverseStack(s);
        insertAtBottom(s, item);
    }
}
```

Functionality:

¿ Takes a pointer to a ‘LogStack‘ as input. Base Case: If the stack is empty, the recursion stops. Recursive Step: If the stack is not empty, it pops the top element, recursively calls ‘reverseStack‘ to reverse the remaining stack, and then calls ‘insertAtBottom‘ to place the popped element at the bottom of the now-reversed smaller stack.

¿

29.10 Displaying a Single Log Entry

The ‘displayLog‘ function is a helper to print the details of a single ‘LogEntry‘.

```
void displayLog(LogEntry *entry) {
    printf("Level %d: %s\n", entry->level, entry->message);
}
```

¿

Functionality:

30 Algorithmic Complexity Analysis (Stacks)

Let N be the maximum capacity of the stack ('MAX$_SIZE$')*and*n*bethecurrentnumberofelementsinthestack.Thestackimp*
sizearray.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

`initializeStack`: $O(1)$ - Constant time to set the initial 'top' value. `isStackEmpty`: $O(1)$ - Constant time to check the 'top' index. `isStackFull`: $O(1)$ - Constant time to check the 'top' index against 'MAX$_SIZE-1$. `pushLog`: $O(1)$ - Constant time to increment 'top', copy data (assuming fixed 'MAX$_LENGTH$'). `popLog` :O(1)$-Constanttimetocopydataanddecrement$'top'. `pe` - It calls 'popLog' n times and 'insertAtBottom' n times. 'insertAtBottom' is $O(n)$, leading to an overall $O(n \times n) = O(n^2)$ complexity. `displayLog`: $O(1)$ - Constant time to print a single entry.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

For most operations: $O(N)$ - The space required is dominated by the fixed-size array 'log' within the 'LogStack' structure, which has a size of 'MAX$_SIZE$'.*Thespacedoesnotgrowwiththenumberofeleme*

31 Comparison of Queues and Stacks

Queues and stacks are both fundamental linear data structures used for storing and managing collections of elements. While they share the characteristic of organizing data in a sequence, their primary difference lies in the order in which elements are accessed and removed. This difference dictates their typical use cases and the implementation of their core operations.

31.1 Fundamental Principles

The core distinction between queues and stacks is their access principle:

- – **Queue**: Follows the **First-In, First-Out (FIFO)** principle. The element that was added first is the first one to be removed. Elements are added at one end (the "rear" or "tail") and removed from the other end (the "front" or "head").
- **Stack**: Follows the **Last-In, First-Out (LIFO)** principle. The element that was added last is the first one to be removed. Elements are added and removed from the same end (the "top").

31.2 Core Operations

The primary operations on queues and stacks reflect their access principles:

- **Queue Operations** (as seen in Chapter 20):
  - `enqueue`: Adds an element to the rear of the queue.
  - `dequeue`: Removes an element from the front of the queue.
  - `peek`: Views the element at the front without removing it.
  - `isEmpty`, `isFull`: Check the state of the queue.
- **Stack Operations** (as seen in Chapter 25):
  - `pushLog`: Adds an element to the top of the stack.
  - `popLog`: Removes an element from the top of the stack.
  - `peekLog`: Views the element at the top without removing it.
  - `isStackEmpty`, `isStackFull`: Check the state of the stack.

As discussed in the complexity analysis sections (Section 25 and 30), the core enqueue/dequeue and push/pop/peek operations typically have a time complexity of $O(1)$ for both array-based and linked-list based implementations (though linked list implementations might have $O(n)$ for operations like adding/removing at the end in a singly linked list).

31.3 Typical Use Cases

The distinct access methods make queues and stacks suitable for different applications:

- **Queue Use Cases**:
  - Managing requests in a shared resource (e.g., printer queue, CPU scheduling).
  - Breadth-First Search (BFS) algorithm.
  - Handling asynchronous data transfer (buffering).
  - Call center systems (handling calls in order).
- **Stack Use Cases**:
  - Function call management (the program's call stack).
  - Expression evaluation and parsing (e.g., converting infix to postfix).
  - Backtracking algorithms (e.g., solving mazes, Sudoku).
  - Undo/Redo functionality in software.
  - Depth-First Search (DFS) algorithm.

31.4 Summary Table

The table below summarizes the key differences between queues and stacks.

Table 6: Comparison of Queues and Stacks

Feature	Queue	Stack
Principle	FIFO (First-In, First-Out)	LIFO (Last-In, First-Out)
Insertion End	Rear (Tail)	Top
Removal End	Front (Head)	Top
Primary Insertion Operation	Enqueue	Push
Primary Removal Operation	Dequeue	Pop
Analogy	Line of people, pipe	Pile of plates, stack of books

In conclusion, while both queues and stacks are linear data structures, their fundamental access principles lead to different operational behaviors and make them appropriate tools for distinct computational problems. The choice between using a queue or a stack depends entirely on the specific requirements of the application and the desired order of element processing.

Recursion

32 Introduction to Recursion

Recursion is a programming technique where a function calls itself in order to solve a problem. A recursive function breaks down a complex problem into smaller, identical subproblems until it reaches a simple base case that can be solved directly. The solutions to the subproblems are then combined to solve the original problem.

Key components of a recursive function are:

- **Base Case**: A condition that stops the recursion. Without a base case, the function would call itself indefinitely, leading to a stack overflow.
- **Recursive Step**: The part of the function where it calls itself with a modified input, moving closer to the base case.

Recursion can often provide elegant and concise solutions for problems that have a naturally recursive structure, such as tree traversals, graph algorithms, and certain mathematical calculations.

33 Recursive Functions in the Log Management System

The ‘recursion.c‘ file demonstrates the use of recursion for various tasks, including mathematical calculations and linked list operations.

33.1 Factorial Calculation

The ‘factorial‘ function calculates the factorial of a non-negative integer recursively.

```
int factorial(int n){
    if (n<0)
    {
        return -1; // Handle negative input
    }
    if (n==0)
    {
        return 1; // Base case: factorial of 0 is 1
    }
    return n*factorial(n-1); // Recursive step
}
```

Functionality:

- **Base Case**: If n is 0, it returns 1.
- **Recursive Step**: If $n > 0$, it returns n multiplied by the factorial of $n - 1$.
- Handles negative input by returning -1.

33.2 Fibonacci Series Calculation

The ‘fibonachiSeries‘ function calculates the n-th term of the Fibonacci series recursively. The Fibonacci series starts with 0 and 1, and each subsequent term is the sum of the two preceding ones (e.g., 0, 1, 1, 2, 3, 5, 8...).

```
int fibonachiSeries(int n){
    if (n<0)
    {
        return -1; // Handle negative input
    }

    if (n==0)
    {
        return 0; // Base case: 0th term is 0
    }
    if (n==1)
    {
        return 1; // Base case: 1st term is 1
    }
    return fibonachiSeries(n-1)+fibonachiSeries(n-2); // Recursive step
}
```

¿

Functionality:

33.3 Recursive Linked List Reversal

The ‘recursion.c‘ file includes functions to reverse a singly linked list using recursion. This involves a helper function ‘ReverseSubFunction‘ and a main function ‘ReverseWithRecursion‘.

Helper Function: ‘ReverseSubFunction‘

This function recursively reverses a linked list and helps update the new head of the reversed list.

```c
LogEntry*ReverseSubFunction(LogEntry*L,LogEntry**New){
    if(L->next==NULL){
        *New=L; // Base case: The last node is the new head
        return L;
    }
    ReverseSubFunction(L->next,New)->next=L; // Recursive step: Reverse the rest
    and link current node
    L->next=NULL; // Set the next of the current node to NULL (it becomes the new
    tail of its sublist)
    return L; // Return the current node
}
```

¿

Functionality:

¿**Base Case**: If the current node ‘L‘ is the last node (its ‘next‘ is ‘NULL‘), it sets the ‘*New‘ pointer (which points to the head pointer in ‘ReverseWithRecursion‘) to this node and returns the node. **Recursive Step**: It recursively calls ‘ReverseSubFunction‘ on the rest of the list (‘L-¿next‘). The recursive call returns the *previous* node in the original list (which is the next node in the reversed list). It then sets the ‘next‘ pointer of that returned node to the current node ‘L‘, effectively linking ‘L‘ into the reversed list. It sets ‘L-¿next‘ to ‘NULL‘ as ‘L‘ will become the tail of the sublist being processed in this step.

¿

Main Function: ‘ReverseWithRecursion‘

This function initiates the recursive reversal process.

```c
void ReverseWithRecursion(LogEntry**L){
    LogEntry*Temp; // Temporary pointer to hold the new head
    ReverseSubFunction(*L,&Temp); // Call the recursive helper
    *L=Temp; // Update the main head pointer
}
```

¿

Functionality:

¿Takes a pointer to the head pointer of the list (‘LogEntry**L‘). Declares a temporary pointer ‘Temp‘. Calls the recursive helper ‘ReverseSubFunction‘, passing the current head and the address of ‘Temp‘. The helper will reverse the list and store the new head’s address in ‘Temp‘. Updates the main head pointer ‘*L‘ to the value stored in ‘Temp‘, making the reversed list the new list.

¿

33.4 Finding Maximum Log ID Recursively

The ‘FindMAxLogId‘ function finds the maximum ‘id‘ among all log entries in a singly linked list using recursion.

```c
int FindMAxLogId(LogEntry*L){
    if (IsLogsEmpty(L))
    {
        return -1; // Base case: Empty list, return error value
    }
    if(L->next==NULL){
        return L->id; // Base case: Single node, return its ID
    }
    // Recursive step: Compare current node's ID with max ID in the rest of the
    list
    return FindMAxLogId(L->next)>L->id ? FindMAxLogId(L->next) : L->id;
}
```

Functionality:

- **Base Cases**: If the list is empty, it returns -1. If the current node is the last node, it returns its ID.
- **Recursive Step**: It recursively calls 'FindMAxLogId' on the rest of the list ('L-¿next') to find the maximum ID in the sublist. It then compares this maximum ID from the sublist with the ID of the current node ('L-¿id') and returns the larger of the two.

34 Algorithmic Complexity Analysis (Recursion)

The complexity of recursive functions is analyzed based on the number of recursive calls and the work done in each call.

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

- `factorial(n)`: $O(n)$ - The function makes n recursive calls, and each call involves constant time operations.
- `fibonachiSeries(n)`: $O(2^n)$ - This is a highly inefficient recursive implementation due to repeated calculations of the same Fibonacci numbers. The number of calls grows exponentially with n. An iterative approach or memoization can reduce this to $O(n)$.
- `ReverseSubFunction(L, New)` and `ReverseWithRecursion(L)`: $O(n)$ - The recursive helper visits each node in the linked list exactly once to reverse the pointers.
- `FindMAxLogId(L)`: $O(n)$ - The function visits each node in the linked list exactly once to compare IDs.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

- `factorial(n)`: $O(n)$ - Due to the recursive calls, the call stack can grow up to a depth of n.
- `fibonachiSeries(n)`: $O(n)$ - The depth of the recursion tree can go up to n.
- `ReverseSubFunction(L, New)` and `ReverseWithRecursion(L)`: $O(n)$ - The depth of the recursion is proportional to the number of nodes in the list.
- `FindMAxLogId(L)`: $O(n)$ - The depth of the recursion is proportional to the number of nodes in the list.

Summary of Recursive Function Complexities:

Table 7: Recursive Function Complexities

Function	Time Complexity (Worst Case)	Space Complexity
factorial(n)	$O(n)$	$O(n)$
fibonachiSeries(n)	$O(2^n)$	$O(n)$
ReverseWithRecursion (on a list of size n)	$O(n)$	$O(n)$
FindMAxLogId (on a list of size n)	$O(n)$	$O(n)$

Binary Search Trees

35 Introduction to Trees

Trees are non-linear data structures that organize data in a hierarchical manner. They consist of nodes connected by edges. A tree has a root node, and each node can have zero or more child nodes. Unlike linear structures like linked lists, trees allow for more efficient searching, insertion, and deletion in many cases, depending on the type of tree.

36 Introduction to Binary Search Trees

A Binary Search Tree (BST) is a special type of binary tree where for each node:

- All nodes in the left subtree have a key (value) less than the node's key.
- All nodes in the right subtree have a key (value) greater than the node's key.
- Both the left and right subtrees are also Binary Search Trees.

This ordering property makes BSTs particularly efficient for searching, as you can quickly eliminate half of the remaining tree at each step based on the comparison with the current node's key. The 'trees.c' implementation uses the timestamp of the log entry as the key for ordering nodes in the BST.

37 Log Entry Structure (Binary Search Tree)

The 'trees.c' file defines a specific structure for nodes in the Binary Search Tree, named 'BsTLogEntry'. This structure includes pointers to both left and right children, in addition to the log entry data.

```
struct BsTLogEntry
{
    struct BsTLogEntry *left;
    int id;
    char message[100];
    struct BsTLogEntry *right;
    char severity[10];
    char timestamp[50];
};
typedef struct BsTLogEntry BsTLogEntry;
```

Structure Breakdown:

- 'left': A pointer to the left child node. According to BST properties, this child and its subtree will contain log entries with timestamps lexicographically less than the current node's timestamp.
- 'id': An integer identifier for the log entry.
- 'message': A character array to store the log message content.
- 'right': A pointer to the right child node. This child and its subtree will contain log entries with timestamps lexicographically greater than or equal to the current node's timestamp (based on the 'ADDNode' implementation's comparison).
- 'severity': A character array for the log's severity level.
- 'timestamp': A character array storing the log's timestamp, used as the key for BST ordering.

38 Binary Search Tree Implementation Details

38.1 Creating a New BST Log Entry Node

The 'CreateBstLogEntry' function allocates memory for a new 'BsTLogEntry' node and initializes its fields.

```
BsTLogEntry*CreateBstLogEntry(){
    BsTLogEntry* temp = (BsTLogEntry*)malloc(sizeof(BsTLogEntry));
    if (temp == NULL) {
        printf("\nMemory Allocation Failed\n");
        return NULL;
    }

    temp->id = ID++; // Assuming ID is a global counter
    temp->right = NULL;
    temp->left =NULL; // Initialize child pointers to NULL
    char msg[100];
    char msg2[100];
    char* temp3 = getCurrentTimeString(); // Function to get timestamp

```

```
    if (temp3) {
        strcpy(temp->timestamp, temp3);
        free(temp3);
    } else {
        strcpy(temp->timestamp, "ERROR");
    }

    printf("\nWhat is the message?\n");
    fgets(msg2, sizeof(msg2), stdin);
    size_t len = strlen(msg2);
    if (len > 0 && msg2[len - 1] == '\n') {
        msg2[len - 1] = '\0';
    }
    strcpy(temp->message, msg2);

    do {
        printf("What is the level of severity (INFO, WARNING, ERROR)?\n");
        if (fgets(msg, sizeof(msg), stdin) != NULL) {
            size_t len = strlen(msg);
            if (len > 0 && msg[len - 1] == '\n') {
                msg[len - 1] = '\0';
            }
            strcpy(temp->severity, msg);
        } else {
            printf("Error reading severity level.\n");
            free(temp);
            return NULL;
        }
    } while (strcmp(temp->severity, "INFO") != 0 && strcmp(temp->severity, "WARNING
    ") != 0 && strcmp(temp->severity, "ERROR") != 0);
    return temp;
}
```

Functionality:

¿Allocates memory for a new ‘BsTLogEntry‘ node using ‘malloc‘. Assigns a unique ID (assuming a global ‘ID‘ counter). Initializes ‘left‘ and ‘right‘ child pointers to ‘NULL‘. Gets the current timestamp and copies it to the ‘timestamp‘ field. Prompts the user for the log message and severity level, reading and validating input. Returns the newly created node or ‘NULL‘ on memory allocation or input failure.

¿

38.2 Adding a Node to the BST (Recursive Helper)

The ‘ADDNode‘ function is a recursive helper function used to find the correct position to insert a new node in the BST based on its timestamp.

```
void ADDNode(BsTLogEntry* B,BsTLogEntry*New) {
    if (B==NULL)
    {
        return; // Should not happen if called correctly from AddLogEntryToBst
    }

    if (strcmp(New->timestamp,B->timestamp)>=0) { // Compare timestamps
        if (B->right == NULL) {
            B->right = New; // Insert as right child
            return;
        } else {
            ADDNode((B->right),New); // Recurse on the right subtree
        }
    } else {
        if (B->left == NULL) {
            B->left = New; // Insert as left child
            return;
        } else {
            ADDNode((B->left),New); // Recurse on the left subtree
        }
    }
}
```

¿

Functionality:

¿Takes the current node ‘B‘ and the new node ‘New‘ to be added as input. Compares the timestamp of the ‘New‘ node with the timestamp of the current node ‘B‘ using ‘strcmp‘. If the new timestamp is greater than or equal to the current node’s timestamp, it checks the right child. If the right child is ‘NULL‘, it inserts the ‘New‘ node there. Otherwise, it recursively calls ‘ADDNode‘ on the right subtree. If the new timestamp is less than the current node’s timestamp, it checks the left child. If the left child is ‘NULL‘, it inserts the ‘New‘ node there. Otherwise, it recursively calls ‘ADDNode‘ on the left subtree.

¿

38.3 Adding a Log Entry to the BST

The ‘AddLogEntryToBst‘ function is the main function for adding a new log entry node to the BST.

```
void AddLogEntryToBst(BsTLogEntry**root,BsTLogEntry*new){
    if (*root==NULL)
    {
        *root=new; // If BST is empty, the new node becomes the root
        return ;
    }
    ADDNode(*root,new); // Use the recursive helper to find insertion point
}
```

¿

Functionality:

¿Takes a pointer to the root pointer of the BST (‘BsTLogEntry**root‘) and the new node ‘new‘ to be added as input. If the BST is currently empty (‘*root‘ is ‘NULL‘), the new node becomes the root. If the BST is not empty, it calls the ‘ADDNode‘ helper function to recursively find the correct position for the new node and insert it.

¿

38.4 Displaying a Single BST Log Entry Node

The ‘displayBsTLogEntryColumn‘ function is a helper to print the details of a single ‘BsTLogEntry‘ node in a formatted way, similar to the linked list display.

```
void displayBsTLogEntryColumn(BsTLogEntry *entry) {
    printf("%-5d | %-32s | %-10s | %-100s\n",
           entry->id, entry->timestamp, entry->severity, entry->message);
}
```

¿

Functionality:

¿Takes a pointer to a ‘BsTLogEntry‘ node as input. Prints the ‘id‘, ‘timestamp‘, ‘severity‘, and ‘message‘ fields of the node in a columnar format.

¿

38.5 Finding the Minimum Node in a Subtree

The ‘findMinBstNode‘ function finds the node with the minimum key (timestamp) in a given subtree. In a BST, the minimum key is always the leftmost node in the subtree.

```
BsTLogEntry* findMinBstNode(BsTLogEntry* node) {
    BsTLogEntry* current = node;

    while (current && current->left != NULL) {
        current = current->left; // Traverse left until the leftmost node is found
    }
    return current; // Return the node with the minimum key
}
```

¿

Functionality:

¿Takes a pointer to the root of the subtree (‘node‘) as input. Starts from the given node and repeatedly moves to the left child as long as the left child is not ‘NULL‘. Returns the pointer to the leftmost node in the subtree, which has the minimum timestamp.

¿

38.6 Deleting a Node from the BST (Recursive)

The ‘DelBsTLogEntryNodeRecursive‘ function is a recursive helper function to delete a node with a specific timestamp from the BST.

```
BsTLogEntry* DelBsTLogEntryNodeRecursive(BsTLogEntry* root, char target_timestamp
    []) {

    if (root == NULL) {
        printf("Node with timestamp '%s' not found.\n", target_timestamp);
        return root; // Base case: Node not found
    }

    int cmp = strcmp(target_timestamp, root->timestamp);

    // Traverse the tree to find the node to delete
    if (cmp < 0) {
        root->left = DelBsTLogEntryNodeRecursive(root->left, target_timestamp); //
    Recurse left
    }
    else if (cmp > 0) {
        root->right = DelBsTLogEntryNodeRecursive(root->right, target_timestamp);
    // Recurse right
    }

    // Node with target_timestamp found
    else {
        // Case 1: Node with only one child or no child
        if (root->left == NULL) {
            BsTLogEntry* temp = root->right;
            free(root); // Free the node
            printf("Node with timestamp '%s' deleted.\n", target_timestamp);
            return temp; // Return the right child (or NULL if no child)
        } else if (root->right == NULL) {
            BsTLogEntry* temp = root->left;
            free(root); // Free the node
            printf("Node with timestamp '%s' deleted.\n", target_timestamp);
            return temp; // Return the left child
        }

        // Case 2: Node with two children
        // Get the in-order successor (smallest in the right subtree)
        BsTLogEntry* temp = findMinBstNode(root->right);

        // Copy the in-order successor's data to this node
        root->id = temp->id;
        strncpy(root->message, temp->message, sizeof(root->message) - 1);
        root->message[sizeof(root->message) - 1] = '\0';
        strncpy(root->severity, temp->severity, sizeof(root->severity) - 1);
        root->severity[sizeof(root->severity) - 1] = '\0';
        strncpy(root->timestamp, temp->timestamp, sizeof(root->timestamp) - 1);
        root->timestamp[sizeof(root->timestamp) - 1] = '\0';

        // Delete the in-order successor from the right subtree
        root->right = DelBsTLogEntryNodeRecursive(root->right, temp->timestamp);
    }
    return root; // Return the (potentially updated) root of the subtree
}
```

Functionality:

¿Takes the current root of the subtree and the target timestamp as input. **Base Case**: If the current root is ‘NULL‘, the node is not found, and it returns ‘NULL‘. It compares the

target timestamp with the current node's timestamp. If the target is smaller, it recursively calls on the left subtree. If the target is larger, it recursively calls on the right subtree. If the target timestamp matches the current node's timestamp, the node to be deleted is found. It handles three deletion cases:

- * Node with no children or one child: The node is freed, and its child (or 'NULL') is returned to link with the parent.
  * Node with two children: The in-order successor (the smallest node in the right subtree) is found. The data from the in-order successor is copied to the current node. Then, the in-order successor is recursively deleted from the right subtree.

¿

- Prints a message indicating whether the node was found and deleted.
- Returns the root of the modified subtree.

¿

38.7 Deleting a Node from the BST

The 'DelBsTLogEntryNode' function is the main function to initiate the recursive deletion process from the BST.

```
void DelBsTLogEntryNode(BsTLogEntry** root, char target_timestamp[]) {
    if (*root == NULL) {
        printf("BST is empty. Cannot delete.\n");
        return;
    }
    *root = DelBsTLogEntryNodeRecursive(*root, target_timestamp); // Call the
    recursive helper
}
```

¿

Functionality:

¿Takes a pointer to the root pointer of the BST and the target timestamp as input. Checks if the BST is empty. Calls the recursive helper function 'DelBsTLogEntryNodeRecursive' to perform the deletion, updating the root pointer if the root node is deleted.

¿

38.8 BST Traversal: In-Order

The 'PrintBsTLogEntryInOrder' function performs an in-order traversal of the BST. In-order traversal of a BST visits nodes in ascending order of their keys (timestamps).

```
void PrintBsTLogEntryInOrder(BsTLogEntry*B){
    if (B==NULL)
    {
        return; // Base case: Empty subtree
    }
    PrintBsTLogEntryInOrder(B->left); // Traverse left subtree
    displayBsTLogEntryColumn(B); // Visit current node
    PrintBsTLogEntryInOrder(B->right); // Traverse right subtree
}
```

¿

Functionality:

¿**Base Case**: If the current node 'B' is 'NULL', the recursion stops. **Recursive Step**: It recursively calls itself on the left subtree, then visits and prints the current node's data, and finally recursively calls itself on the right subtree.

¿

38.9 BST Traversal: Pre-Order

The ‘PrintBsTLogEntryPreOrder‘ function performs a pre-order traversal of the BST. In pre-order traversal, the current node is visited before its children.

```
void PrintBsTLogEntryPreOrder(BsTLogEntry*B){
    if (B==NULL)
    {
        return; // Base case: Empty subtree
    }
    displayBsTLogEntryColumn(B); // Visit current node
    PrintBsTLogEntryPreOrder(B->left); // Traverse left subtree
    PrintBsTLogEntryPreOrder(B->right); // Traverse right subtree
}
```

¿

Functionality:

¿**Base Case**: If the current node ‘B‘ is ‘NULL‘, the recursion stops. **Recursive Step**: It visits and prints the current node’s data, then recursively calls itself on the left subtree, and finally recursively calls itself on the right subtree.

¿

38.10 BST Traversal: Post-Order

The ‘PrintBsTLogEntryPostOrder‘ function performs a post-order traversal of the BST. In post-order traversal, the current node is visited after its children.

```
void PrintBsTLogEntryPostOrder(BsTLogEntry*B){
    if (B==NULL)
    {
        return; // Base case: Empty subtree
    }
    PrintBsTLogEntryPostOrder(B->left); // Traverse left subtree
    PrintBsTLogEntryPostOrder(B->right); // Traverse right subtree
    displayBsTLogEntryColumn(B); // Visit current node
}
\end{lstlisting>

\textbf{Functionality}:
\begin{itemize}>
    \item \textbf{Base Case}: If the current node ‘B‘ is ‘NULL‘, the recursion
    stops.
    \item \textbf{Recursive Step}: It recursively calls itself on the left subtree,
     then recursively calls itself on the right subtree, and finally visits and
    prints the current node’s data.
\end{itemize}>

\subsection{Creating a BST Node from a LogEntry Copy}
\label{ssub:create_node_cpy_to_bst}
The ‘CreateNodeCpyToBst‘ function creates a new ‘BsTLogEntry‘ node and copies data
    from an existing ‘LogEntry‘ (from a linked list).

\begin{lstlisting}
BsTLogEntry*CreateNodeCpyToBst(LogEntry*original){
    if (original==NULL)
    {
        return NULL; // Handle NULL input
    }
    BsTLogEntry*root=malloc(sizeof(BsTLogEntry));
    if (root==NULL)
    {
        return root; // Handle memory allocation failure
    }

    root->id=original->id;
    root->left=NULL;
    root->right=NULL; // Initialize child pointers
    strncpy(root->timestamp,original->timestamp,sizeof(root->timestamp)-1);
```

```
    root->timestamp[sizeof(root->timestamp)-1]='\0'; // Ensure null-termination
    strncpy(root->severity,original->severity,sizeof(root->severity)-1);
    root->severity[sizeof(root->severity)-1]='\0'; // Ensure null-termination
    strncpy(root->message,original->message,sizeof(root->message)-1);
    root->message[sizeof(root->timestamp)-1]='\0'; // Ensure null-termination
    return root ;
}
```

Functionality:

38.11 Converting a Linked List to a BST

The ‘convertLogEntryToBst‘ function converts a singly linked list of ‘LogEntry‘ nodes into a Binary Search Tree of ‘BsTLogEntry‘ nodes, using the timestamp as the key.

```
BsTLogEntry*convertLogEntryToBst(LogEntry*H){
    if (H==NULL)
    {
        return NULL; // Base case: Empty linked list
    }
    else
    {
        BsTLogEntry*root=NULL; // Initialize the root of the new BST
        while (H!=NULL)
        {
            BsTLogEntry*New=CreateNodeCpyToBst(H); // Create a new BST node from
    the current linked list node
            if (New != NULL) {
                AddLogEntryToBst(&root,New); // Add the new BST node to the BST
            }
            H=H->next; // Move to the next node in the linked list
        }
        return root; // Return the root of the constructed BST
    }
}
```

¿

Functionality:

¿Takes the head pointer of a singly linked list (‘LogEntry*H‘) as input. **Base Case**: If the linked list is empty, it returns ‘NULL‘. Initializes the root of the new BST to ‘NULL‘. Traverses the linked list. For each ‘LogEntry‘ in the linked list:

- ∗ It calls ‘CreateNodeCpyToBst‘ to create a new ‘BsTLogEntry‘ node with copied data.
- ∗ If the new BST node is created successfully, it calls ‘AddLogEntryToBst‘ to insert this node into the BST.

¿

- Returns the root of the constructed Binary Search Tree.

¿

38.12 Queue Implementation for Level Order Traversal (Helper)

The ‘trees.c‘ file includes a basic queue implementation (‘struct node‘, ‘Queue‘, ‘createQueue‘, ‘EnQueue‘, ‘DeQueue‘, ‘isEmptyQueue‘). While a full level order traversal function is not explicitly shown using these, this queue structure and its operations are typically used to perform Level Order (Breadth-First) Traversal of a tree.

Node Structure for Queue

```
typedef struct node
{
    BsTLogEntry*val;
    struct node *next;
}node;
```

This structure defines a node for a linked list-based queue, where ‘val‘ stores a pointer to a ‘BsTLogEntry‘ (a BST node).

Queue Structure

```
typedef struct Queue{
    node*front;
    node*rear;
}Queue;
```

This structure represents the queue with pointers to the front and rear nodes.

Queue Operations

```
Queue*createQueue(){
    Queue *Q =malloc(sizeof(Queue));
    Q->front=NULL;
    Q->rear=NULL;
    return Q;
}
void EnQueue(Queue*Q,BsTLogEntry*value){
    node *Temp=NULL;
    if (value==NULL)
    {
        return;
    }

    Temp=malloc(sizeof(node));
    if(Temp==NULL){
        return ;
    }
    Temp->next=NULL;
    Temp->val=value;
    if(Q->front==NULL){
        Q->front=Temp;
            Q->rear=Temp;
        }
        else{
            Q->rear->next=Temp;
            Q->rear=Temp;
        }
}
BsTLogEntry* DeQueue(Queue*Q){
        if(Q->front==NULL){
            return NULL;
        }
        else if(Q->front->next==NULL){
            BsTLogEntry* val=Q->front->val;
            free(Q->front);
            Q->front=NULL;
            Q->rear=NULL;
            return val;
        }
        else{
            BsTLogEntry *val=Q->front->val;
            node * Temp =Q->front->next;
            free(Q->front);
            Q->front=Temp;
            return val;
        }
}
bool isEmptyQueue(Queue*Q){
    return(Q->front==NULL&&Q->rear==NULL);
}
```

Functionality:

¿‘createQueue‘: Allocates memory for and initializes an empty queue. ‘EnQueue‘: Adds a ‘BsTLogEntry‘ pointer to the rear of the queue. ‘DeQueue‘: Removes and returns a ‘BsTLogEntry‘ pointer from the front of the queue. ‘isEmptyQueue‘: Checks if the queue is empty.

¿ These functions suggest the intention to implement level order traversal, which uses a queue to process nodes level by level.

38.13 Finding and Deleting the Minimum Node (Alternative/Helper)

The ‘findMinBstNodeAndDel‘ function finds the node with the minimum key in a subtree and also removes it from the tree. This function seems to be an alternative approach or a helper function for deletion, specifically for finding and detaching the in-order successor.

```
BsTLogEntry* findMinBstNodeAndDel(BsTLogEntry* node) {
    BsTLogEntry* current = node,*prev=NULL;

    while (current && current->left != NULL) {
        prev=current;
        current = current->left; // Traverse left, keeping track of the previous
    node
    }
    // After the loop, current is the minimum node, and prev is its parent
    if (prev != NULL) {
        prev->left=NULL; // Detach the minimum node from its parent
    }
    // Note: This function finds and detaches, but does NOT free the node.
    // The caller is responsible for freeing the returned node and linking it
    correctly.
    return current; // Return the minimum node
}
```

¿

Functionality:

> ¿Takes a pointer to the root of the subtree (‘node‘) as input. Traverses left from the starting node to find the minimum node, keeping track of the ‘prev‘ node (the parent of the current node). Once the minimum node is found, it detaches it from its parent by setting the parent’s left pointer to ‘NULL‘. Returns the pointer to the detached minimum node.

¿ Note that this function only detaches the node; the caller is responsible for freeing the memory of the returned node and correctly integrating it into the deletion process (e.g., as the replacement for a node with two children).

39 Algorithmic Complexity Analysis (Binary Search Trees)

Let n be the number of log entries (nodes) in the Binary Search Tree. The complexity of BST operations depends heavily on the height of the tree, which can vary from $O(\log n)$ for a balanced tree to $O(n)$ for a skewed tree (like a linked list).

Time Complexity

Time complexity measures how the execution time of an algorithm grows with the input size.

- `CreateBstLogEntry`: $O(1)$ - Constant time for allocation and initialization.
- `ADDNode` and `AddLogEntryToBst`: $O(h)$ where h is the height of the tree. In the worst case (skewed tree), $h = O(n)$, so $O(n)$. In the average case (balanced tree), $h = O(\log n)$, so $O(\log n)$.
- `displayBsTLogEntryColumn`: $O(1)$ - Constant time for printing.
- `findMinBstNode`: $O(h)$ - Requires traversing down the leftmost path. Worst case $O(n)$, average case $O(\log n)$.
- `DelBsTLogEntryNodeRecursive` and `DelBsTLogEntryNode`: $O(h)$ - Finding the node is $O(h)$, finding the in-order successor is $O(h)$, and recursive deletion of the successor is also $O(h)$. Worst case $O(n)$, average case $O(\log n)$.
- `PrintBsTLogEntryInOrder`, `PrintBsTLogEntryPreOrder`, `PrintBsTLogEntryPostOrder`: $O(n)$ - Each traversal visits every node exactly once.

- `CreateNodeCpyToBst`: $O(1)$ - Constant time for allocation and copying.
- `convertLogEntryToBst` (from a linked list of size m): $O(m \times h)$ - It iterates through the linked list ($O(m)$) and for each node, inserts it into the BST ($O(h)$). Worst case $O(m \times n)$, average case $O(m \log n)$.
- Queue operations (`createQueue`, `EnQueue`, `DeQueue`, `isEmptyQueue`): $O(1)$ for a linked list based queue implementation.
- `findMinBstNodeAndDel`: $O(h)$ - Traversing to the minimum node. Worst case $O(n)$, average case $O(\log n)$.

Space Complexity

Space complexity measures the amount of memory used by an algorithm.

¿For the BST structure itself: $O(n)$ - The space required is proportional to the number of nodes. Recursive functions (`ADDNode`, `DelBsTLogEntryNodeRecursive`, traversals): $O(h)$ for the recursion stack space. Worst case $O(n)$, average case $O(\log n)$. Queue structure for traversal: $O(w)$ where w is the maximum width of the tree (number of nodes at the widest level). In the worst case (a complete binary tree), $w \approx n/2$, so $O(n)$. `CreateBstLogEntry`, `CreateNodeCpyToBst`: $O(1)$ - Constant extra space per call.

¿

Summary of Binary Search Tree Operation Complexities:

Table 8: Binary Search Tree Operation Complexities

Operation	Time Complexity (Worst Case)	Space Complexity
CreateBstLogEntry	$O(1)$	$O(1)$
AddLogEntryToBst	$O(n)$	$O(n)$
DisplayBsTLogEntryColumn	$O(1)$	$O(1)$
FindMinBstNode	$O(n)$	$O(n)$
DelBsTLogEntryNode	$O(n)$	$O(n)$
In-Order Traversal	$O(n)$	$O(n)$
Pre-Order Traversal	$O(n)$	$O(n)$
Post-Order Traversal	$O(n)$	$O(n)$
CreateNodeCpyToBst	$O(1)$	$O(1)$
Convert Linked List to BST (list size m)	$O(m \times n)$	$O(n)$
Queue Operations (Enqueue, Dequeue, etc.)	$O(1)$	$O(n)$ (for queue size)
FindMinBstNodeAndDel	$O(n)$	$O(n)$

40 Conclusion

The system provides:

- Robust logging capabilities
- Efficient sorting and searching
- Comprehensive memory management
- Thread-safe operations

UX Considerations:

- Consistent prompt formatting
- Input sanitization
- Clear error messages